



Lecture 10:

Image Generation Models

(Part 2)

Email: idadschwartz@gmail.com or
idan.schwartz@biu.ac.il

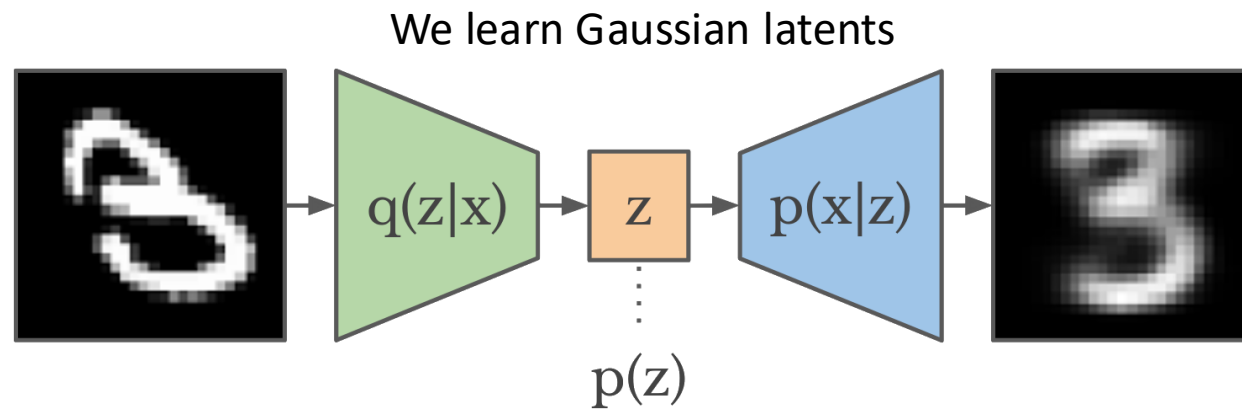
Generating images

- Modeling image distribution is difficult due to the high variability.
- For example, even just for faces, we need to consider factors such as gender, eye color, hair color, and pose
- Those hidden factors are the latents

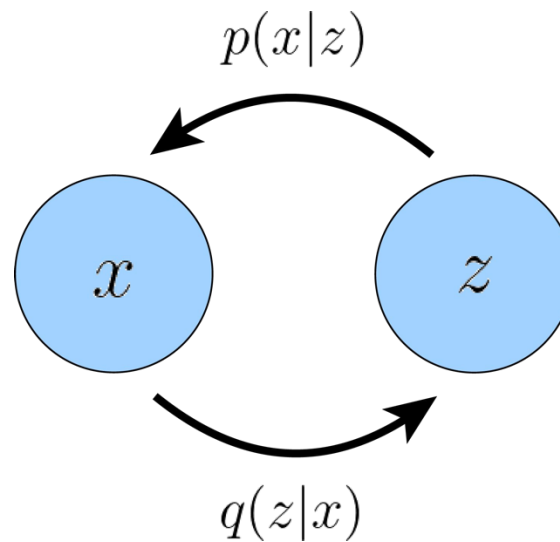
But there are many more variables to consider in natural images



Recall: VAEs



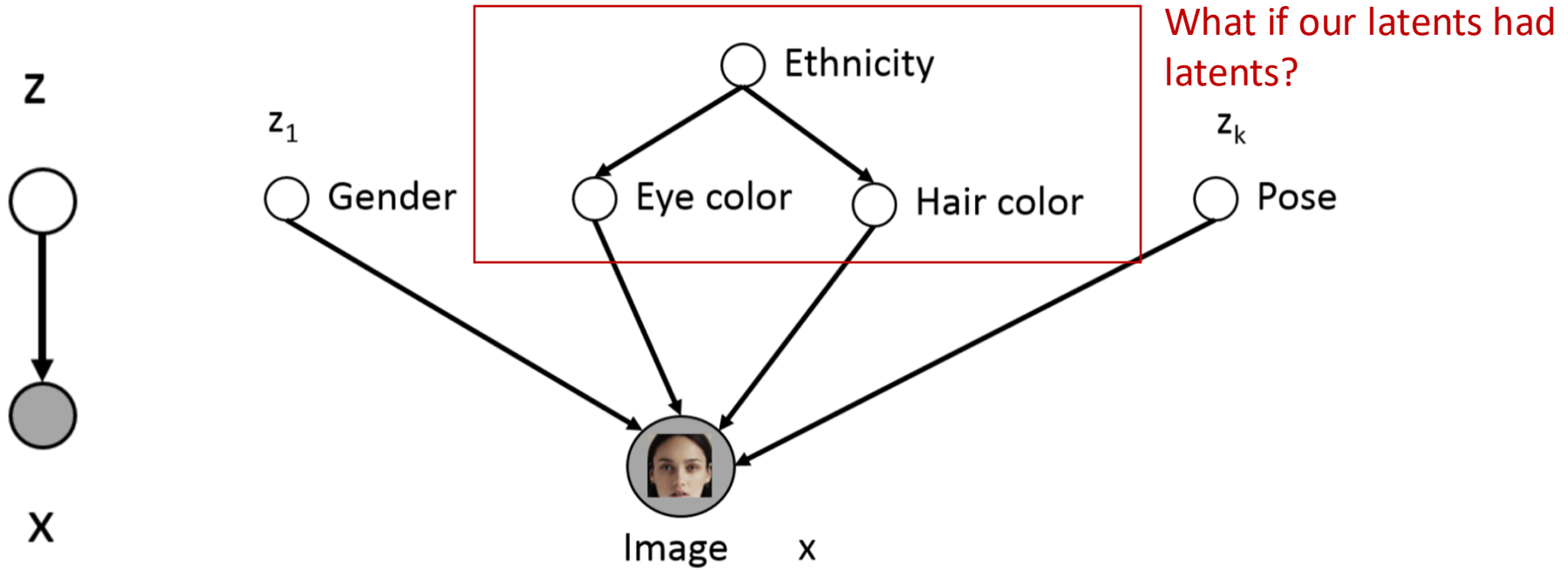
Or as



Recall: diffusion process

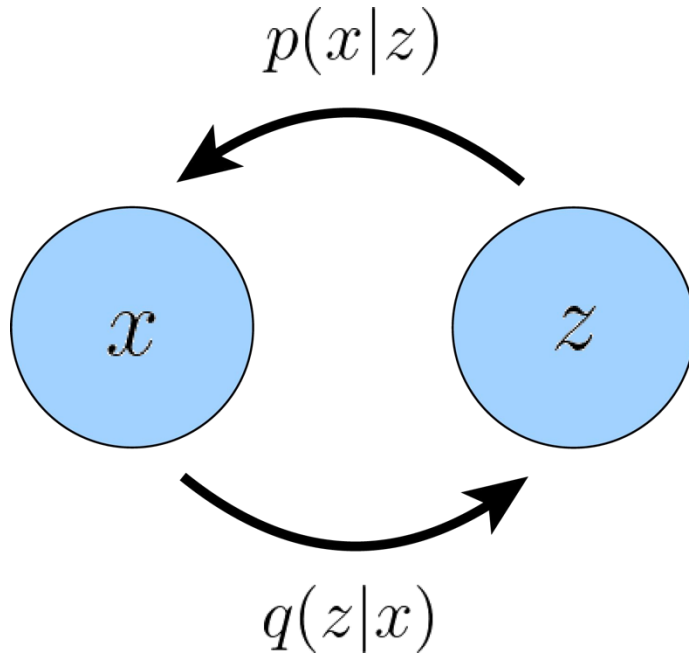


Generative Modeling with Latent Variables



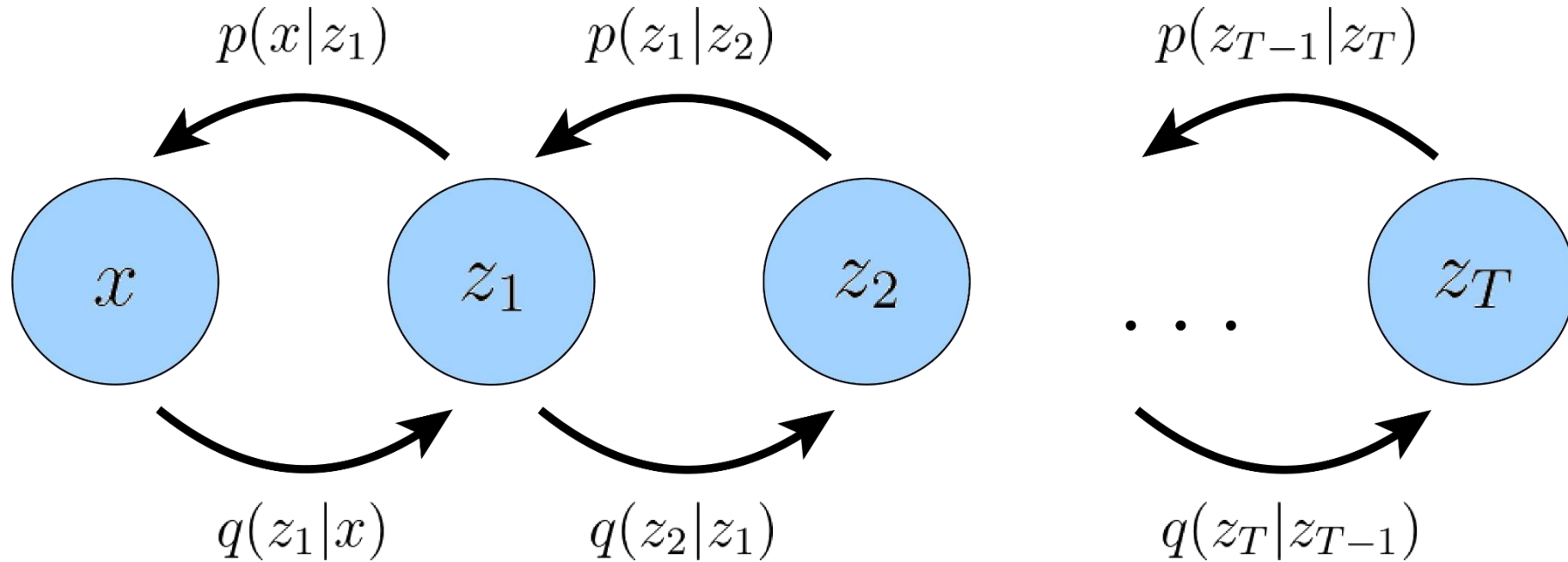
The variational perspective of a diffusion model: Hierarchical VAEs

- Generalize VAEs by enabling a hierarchy of latents $z = z_1, \dots, z_T$

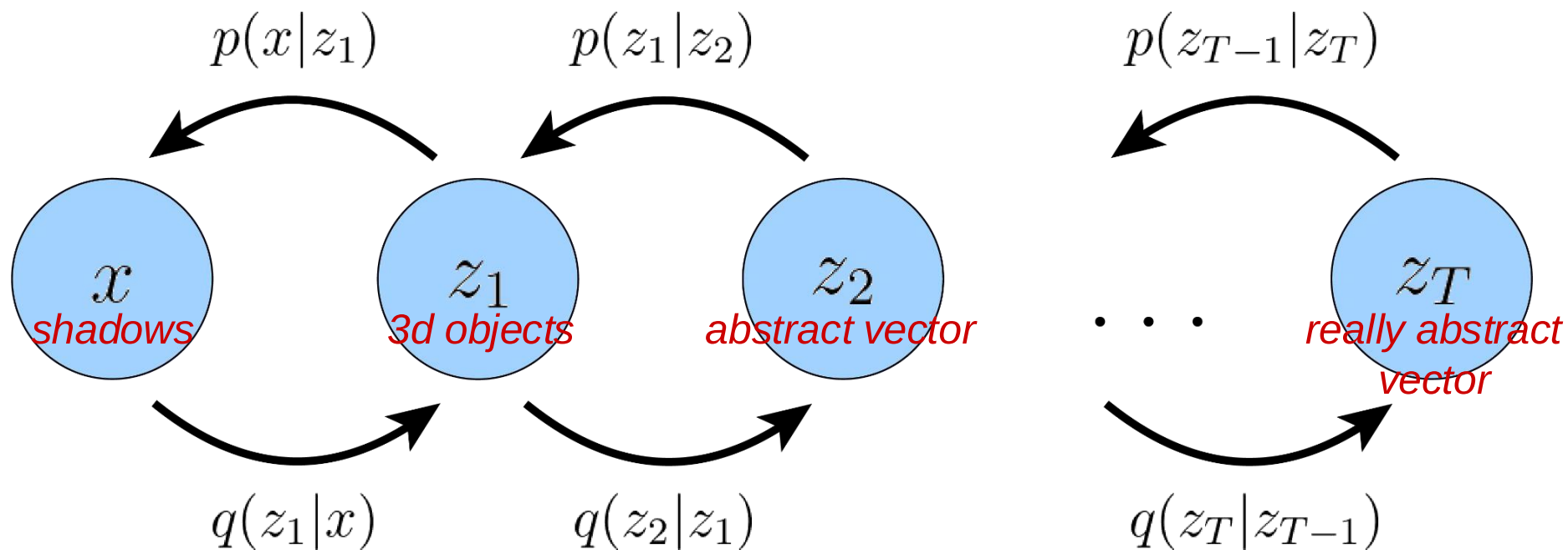


The variational perspective of a diffusion model: Hierarchical VAEs

- Generalize VAEs by enabling a hierarchy of latents $z = z_1, \dots, z_T$

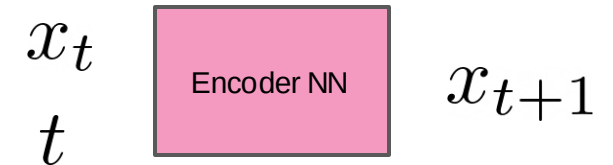
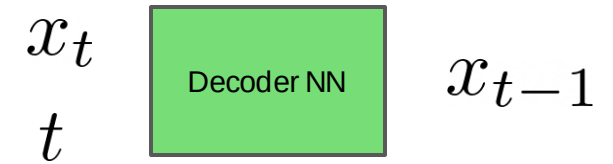
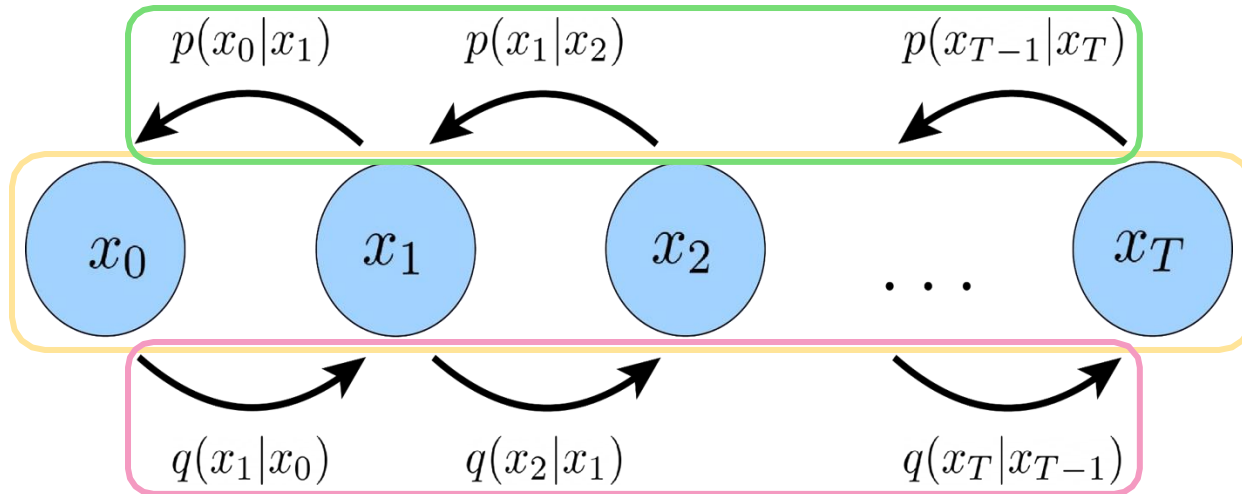


Hierarchical latents intuition



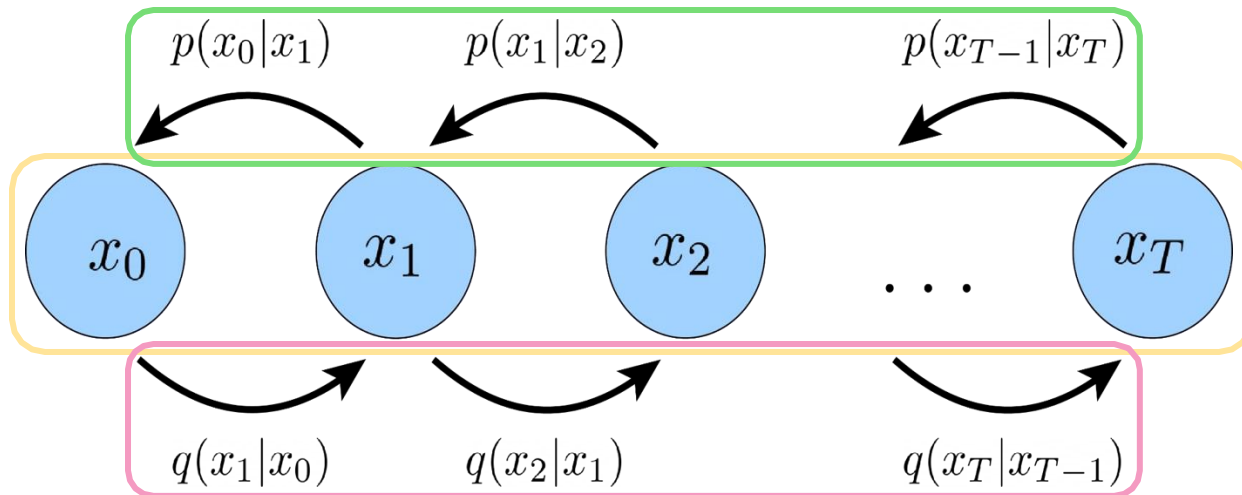
Hierarchical VAEs

- For Hierarchical VAEs we need $t-1$ encoders decoders
- Instead, we assume all dimensions are the same and use a single encoder-decoder

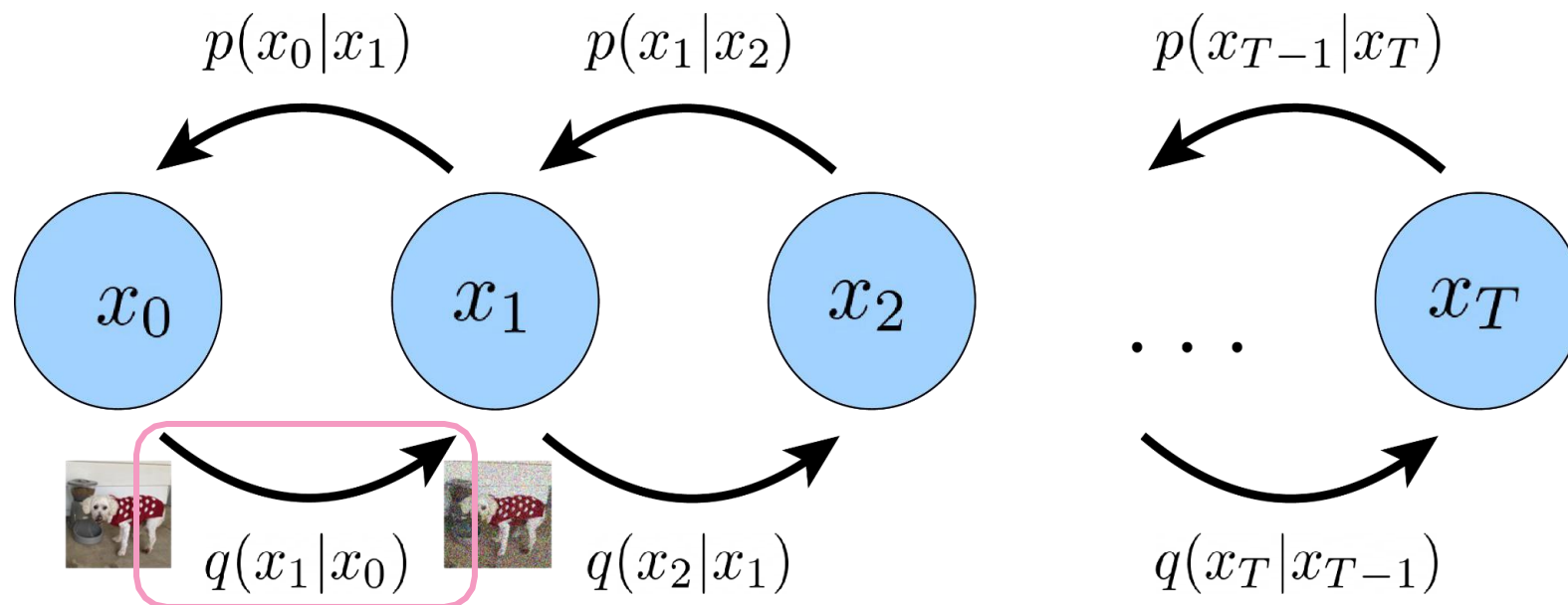


Hierarchical VAEs

- For Hierarchical VAEs we need $t-1$ encoders decoders
- Further, we assume all encoder transitions are known Gaussians centered around their previous input



Let's take a look at one encoding



$$q(x_1|x_0) = \mathcal{N}(x_1|x_0, \sigma_1^2 \mathbf{I})$$



$$x_1 \sim q(x_1|x_0)$$

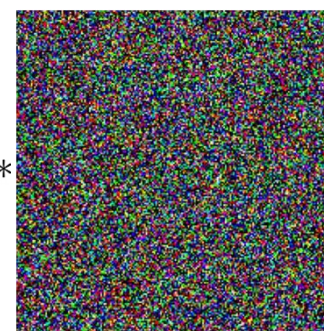
=



$$\mathbf{x}_0$$

+

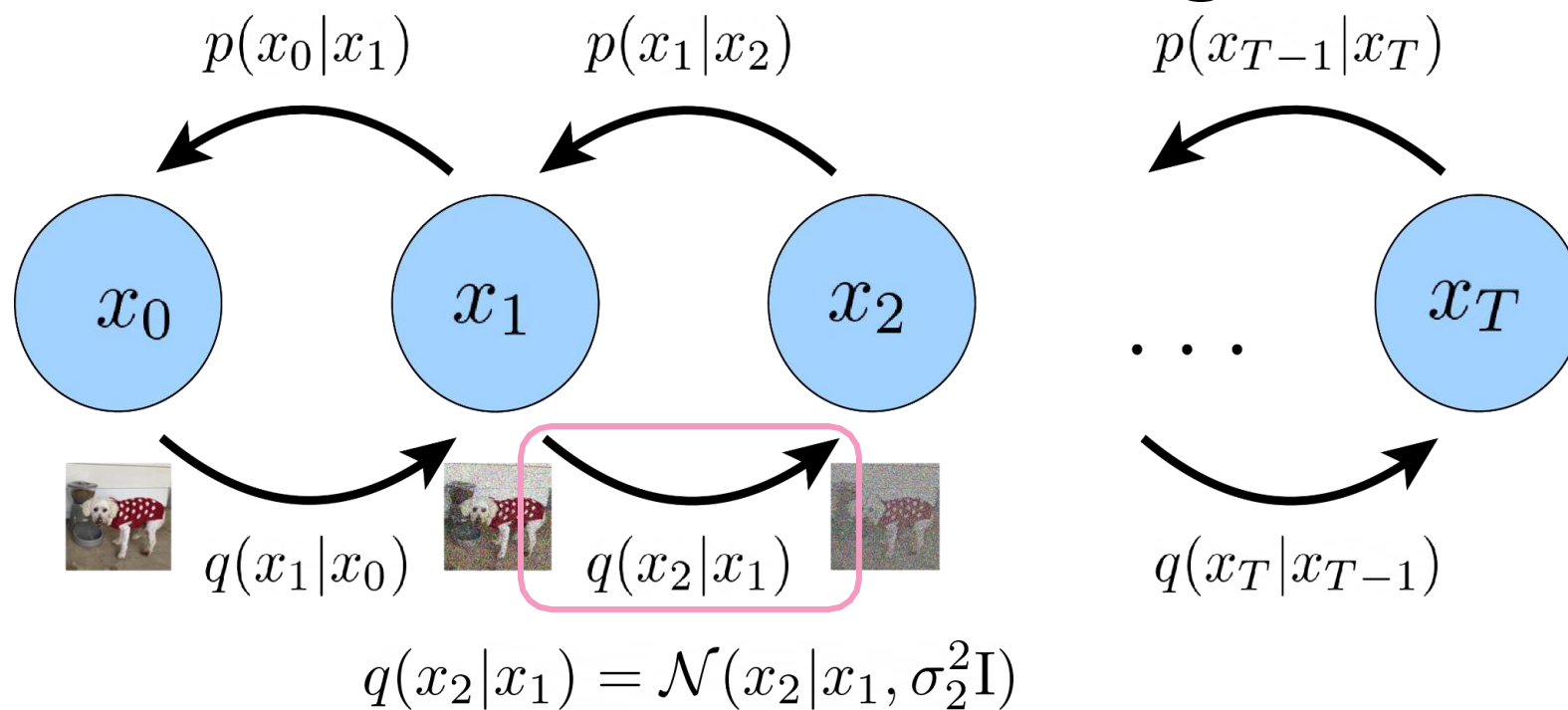
$$\sigma_1 *$$



$$\epsilon_0$$

reparam. trick!

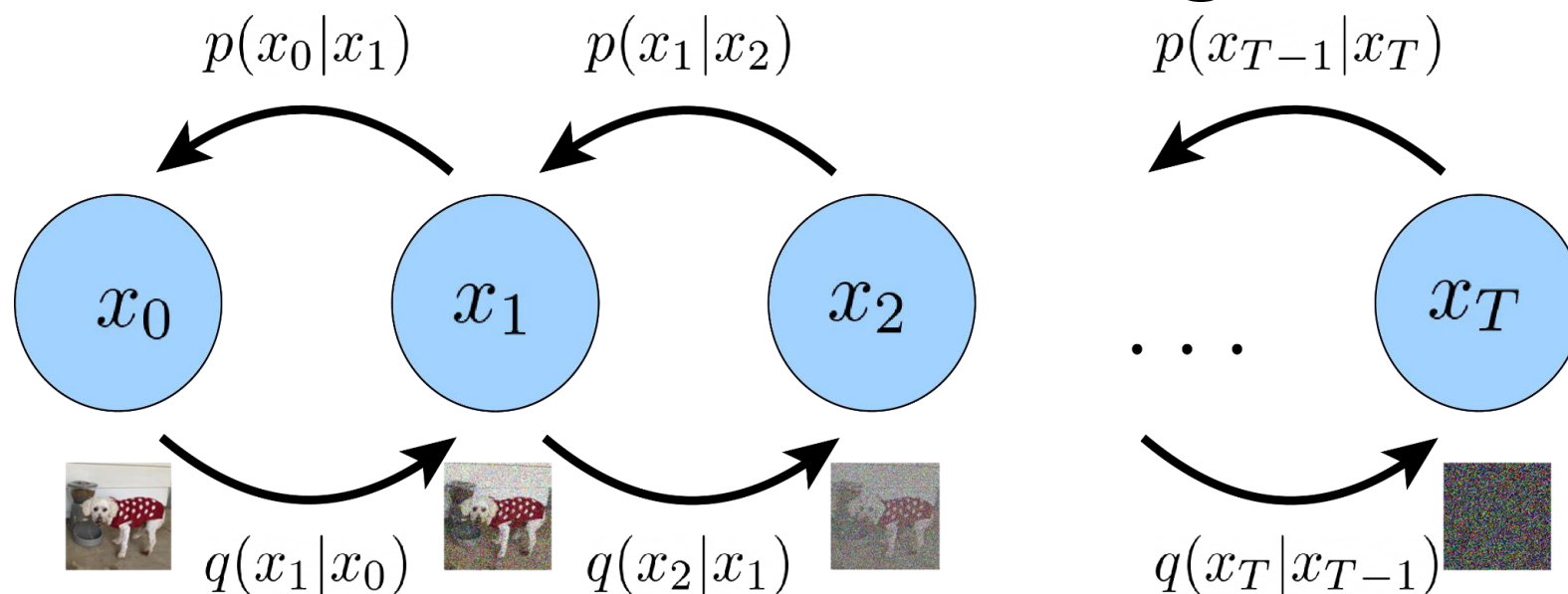
Let's take a look at one encoding



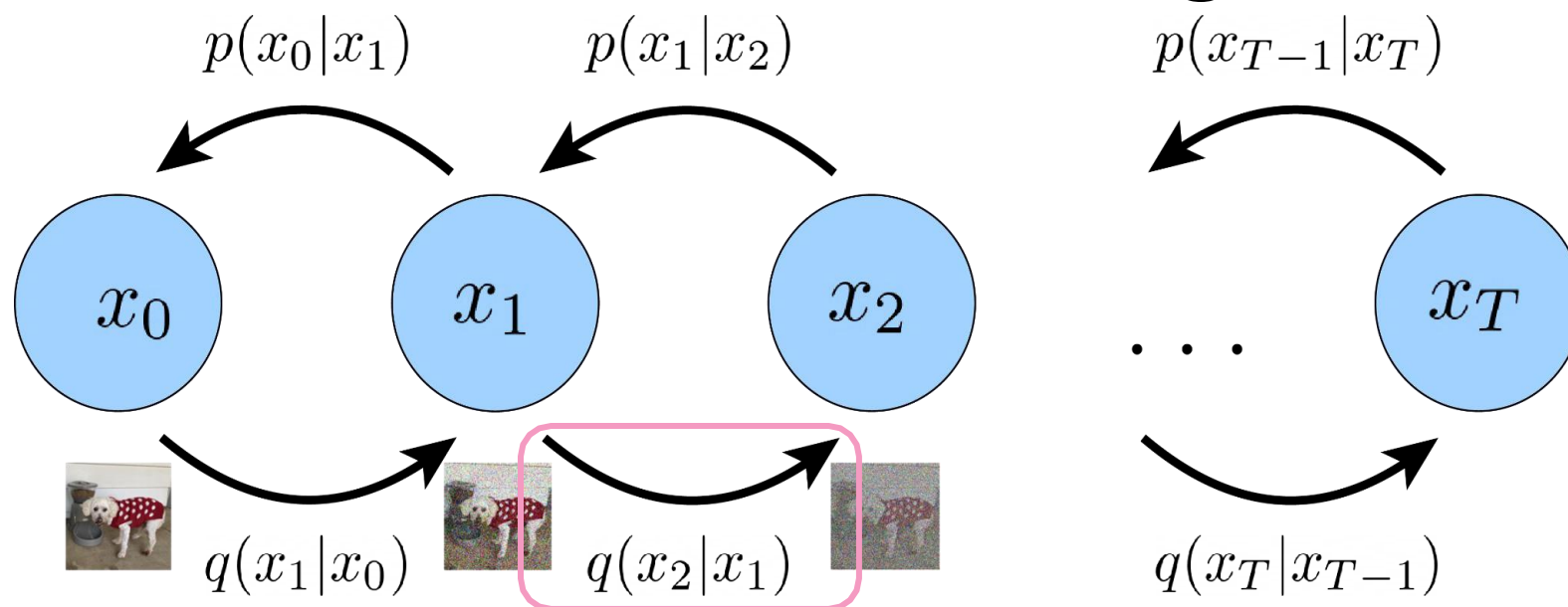
Visual representation of the reparameterization trick. The equation $x_2 \sim q(x_2 | x_1)$ is shown as $x_1 + \sigma_2 * \epsilon_0$, where x_1 is a noisy image of a dog, σ_2 is a scalar, and ϵ_0 is a noise vector.

reparam. trick!

Let's take a look at one encoding



Let's take a look at one encoding

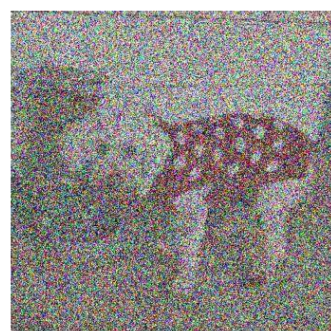
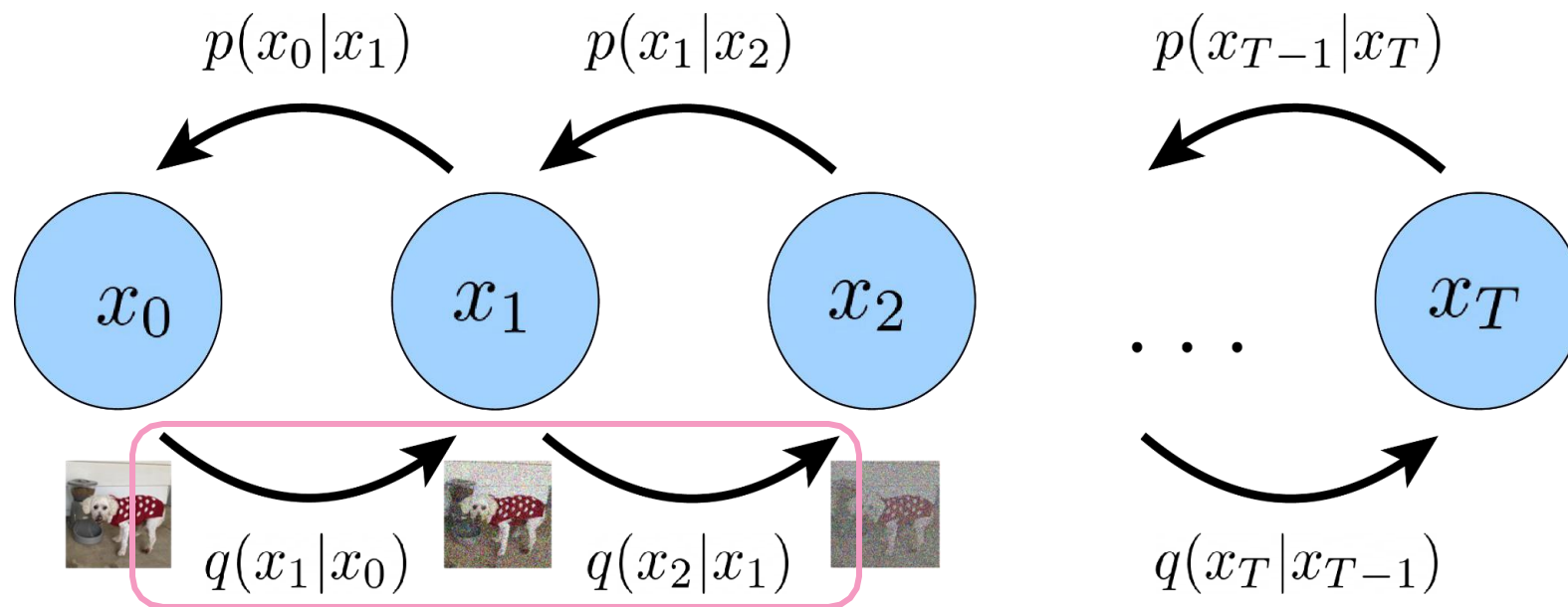


$$x_2 \sim q(x_2|x_1) = x_1 + \sigma_2 \epsilon_0$$

Diagram illustrating the reparameterization trick for sampling x_2 from $q(x_2|x_1)$. The equation shows $x_2 \sim q(x_2|x_1) = x_1 + \sigma_2 \epsilon_0$, where x_1 is the previous latent variable (image of a dog) and ϵ_0 is a noise vector.

reparam. trick!

Let's take a look at one encoding



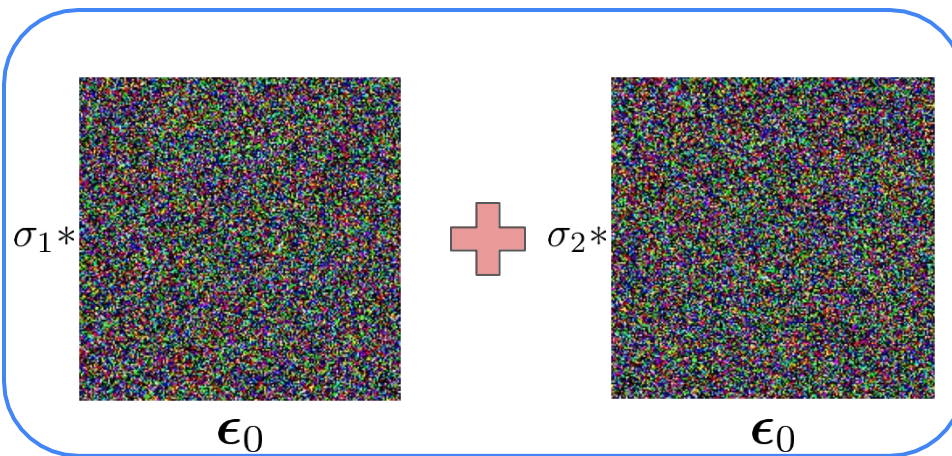
$x_2 \sim q(x_2|x_1)$

=



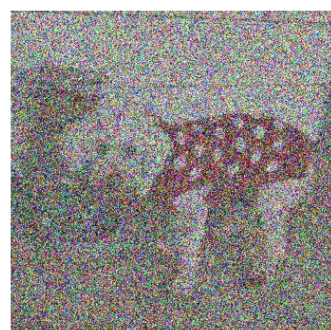
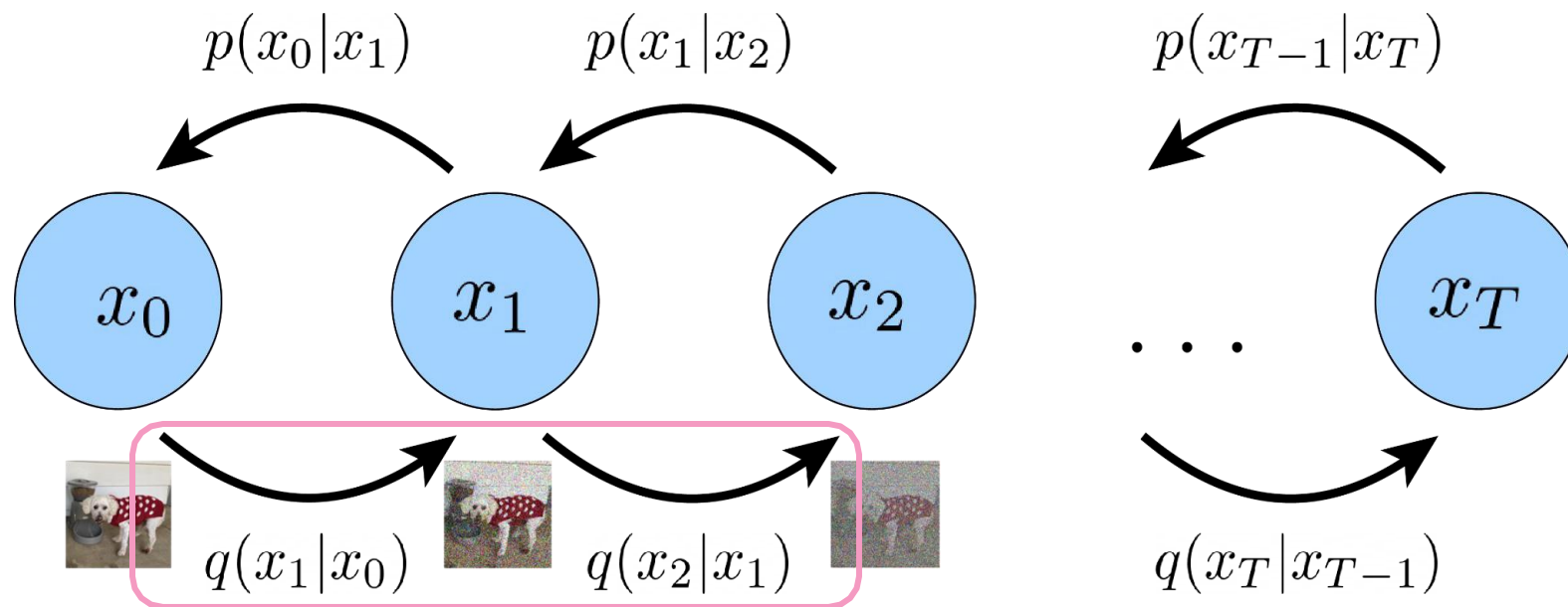
x_0

+



Aggregate into 1 sample!

Let's take a look at one encoding



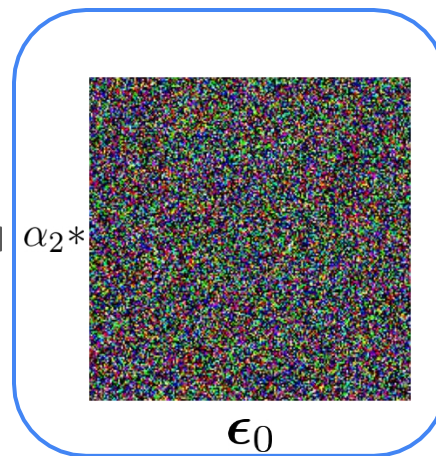
$$x_2 \sim q(x_2|x_1)$$

=



$$x_0$$

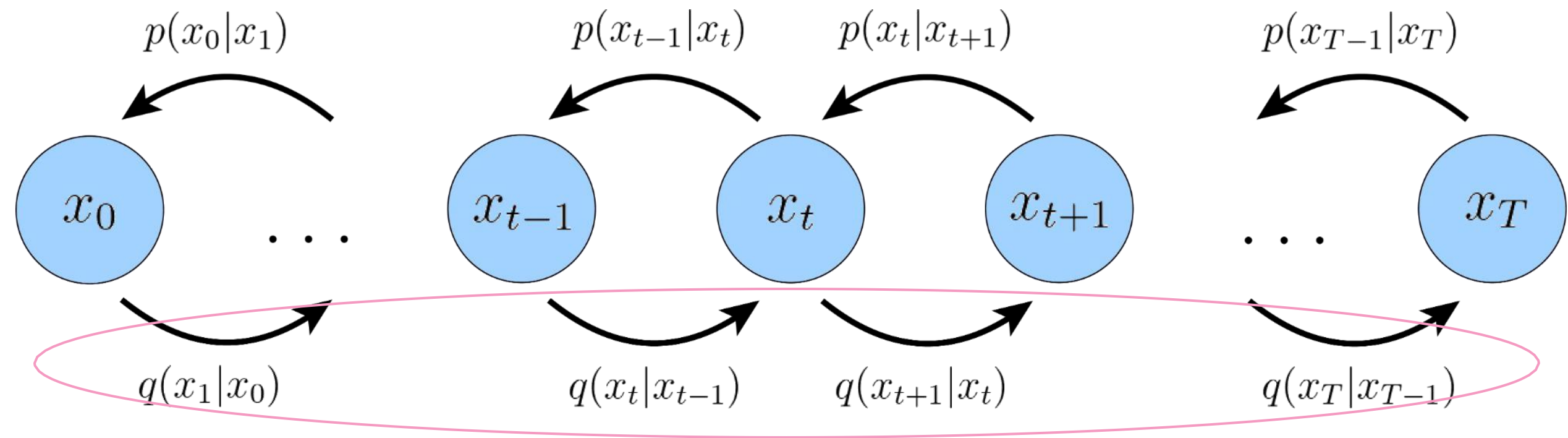
+



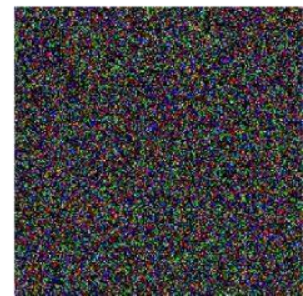
$$\alpha_2 \cdot$$

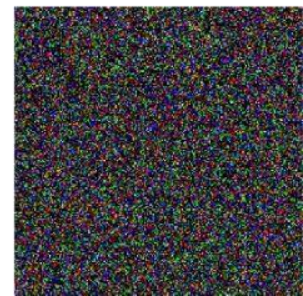
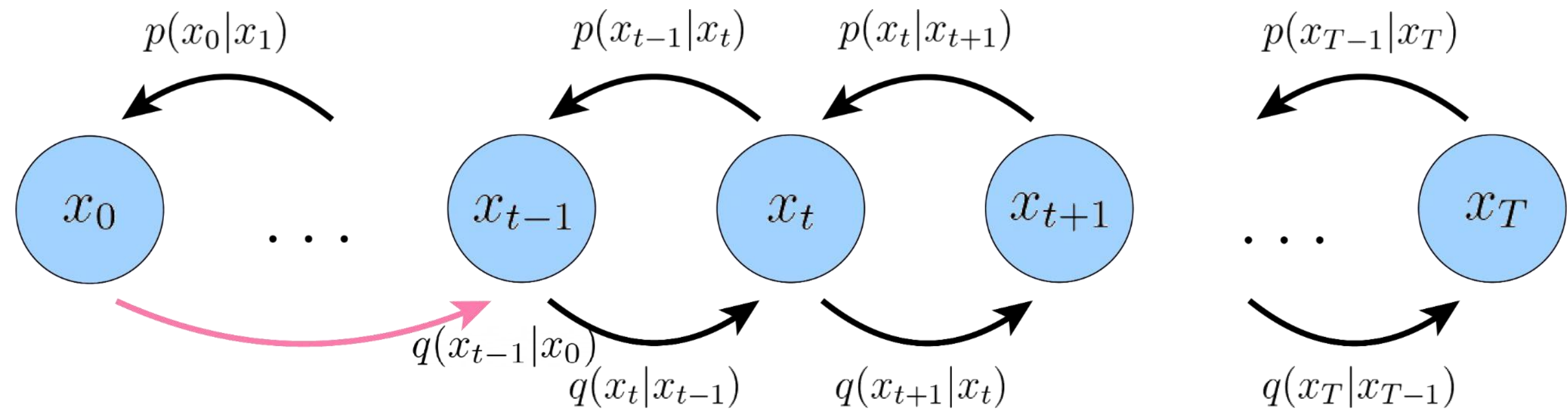
$$\epsilon_0$$

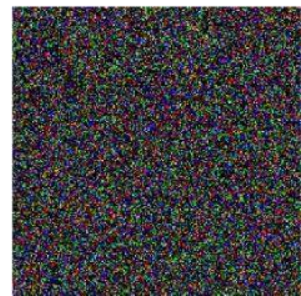
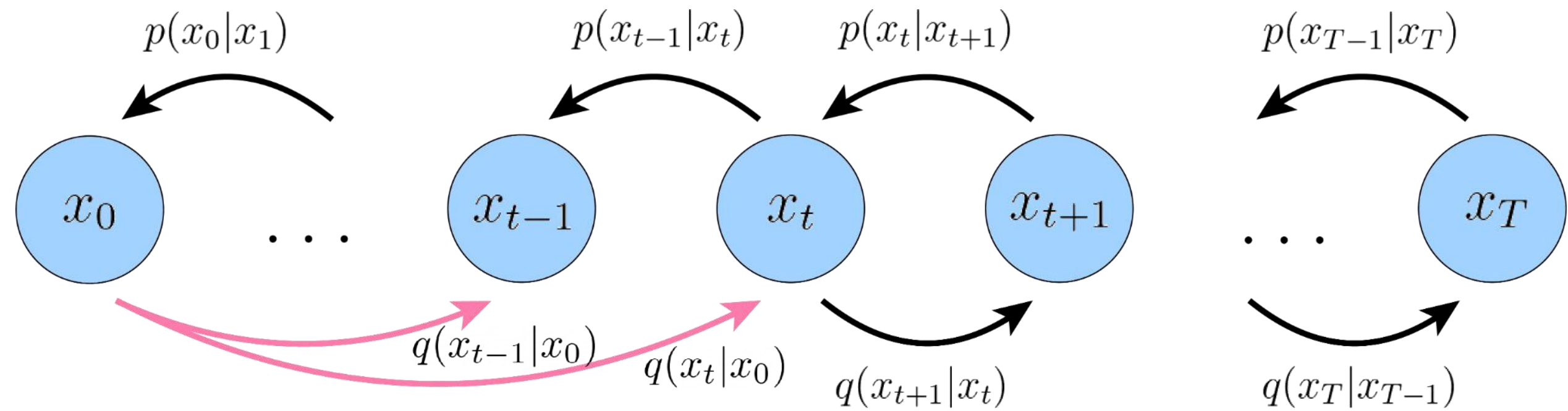
Aggregate into 1 sample!

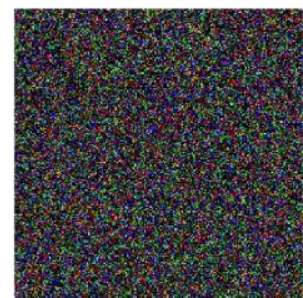
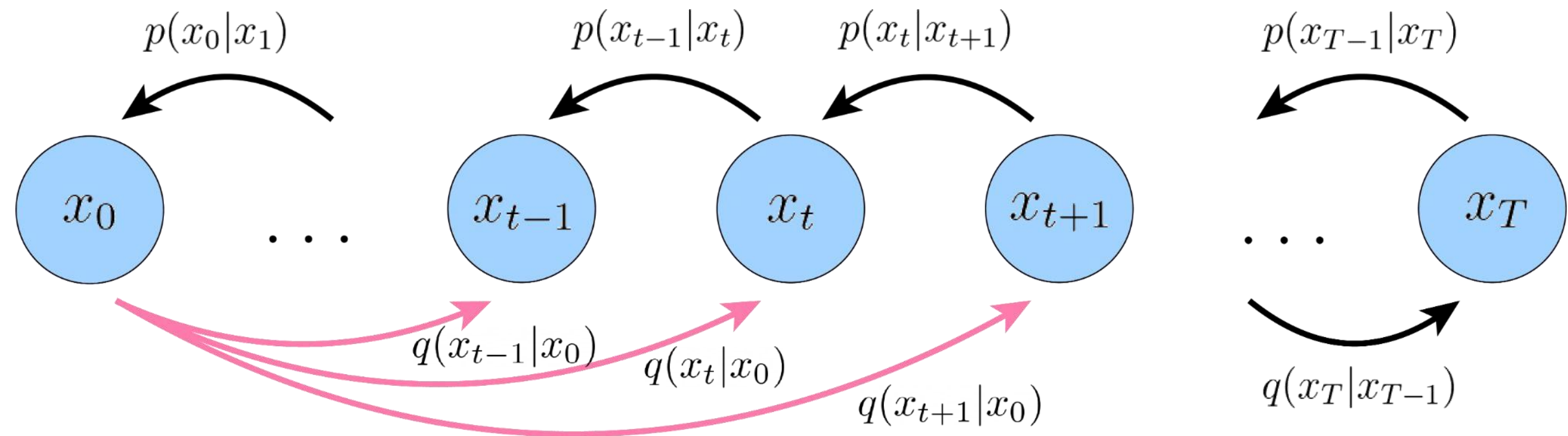


Individual (known) Gaussians!

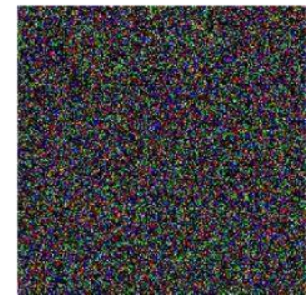
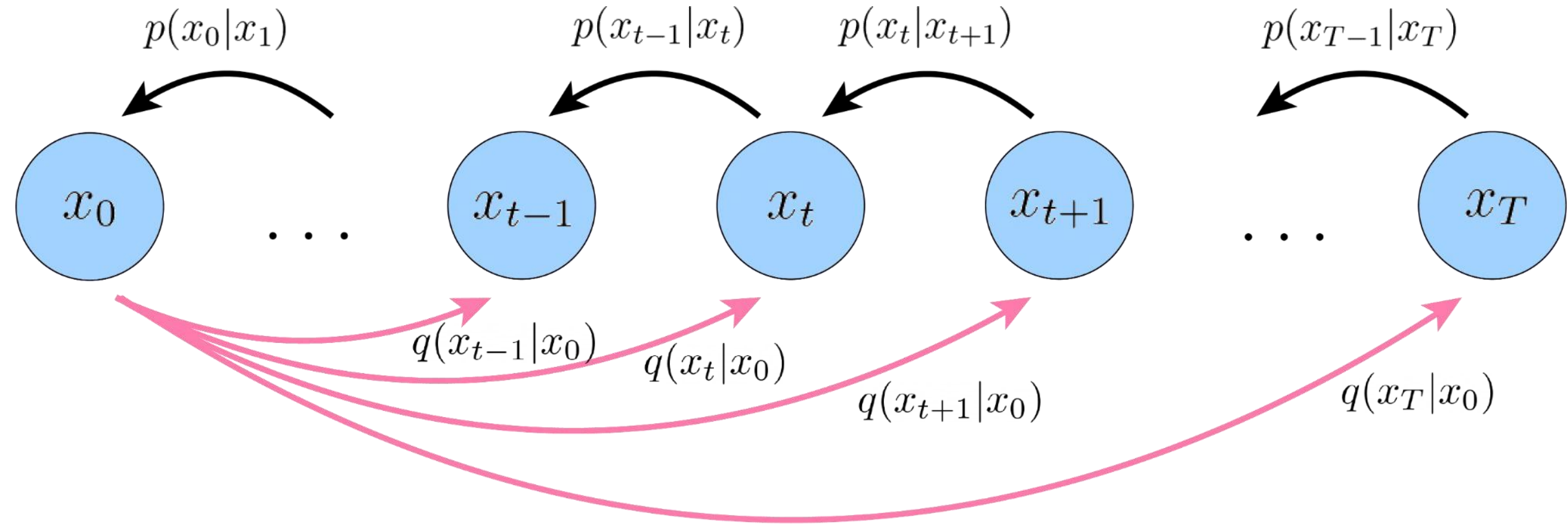




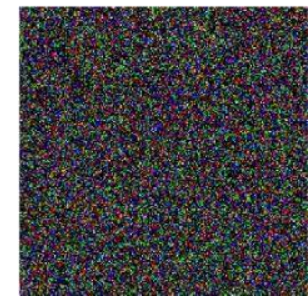
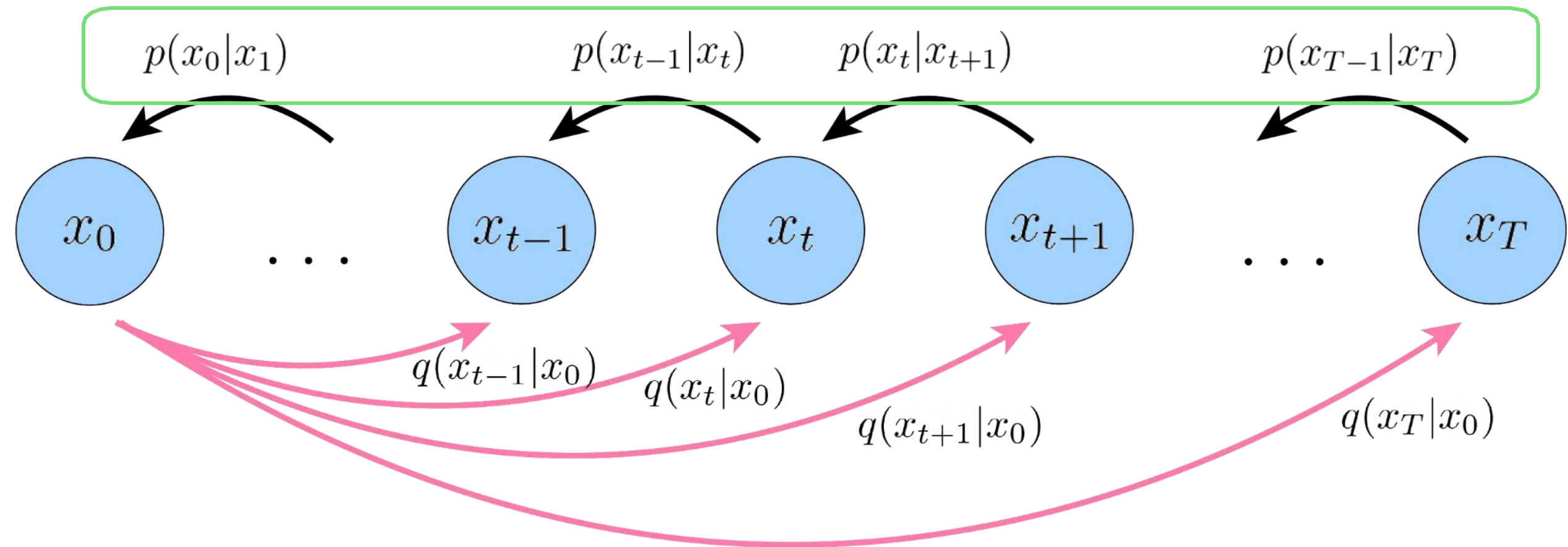




$q(x_t|x_0)$ is a Gaussian, for arbitrary t !
 $q(x_t|x_0) = \mathcal{N}(x_t|x_0, \alpha_t^2 \mathbf{I})$, where $\alpha_0, \alpha_1, \dots, \alpha_T$ are all known/fixed.

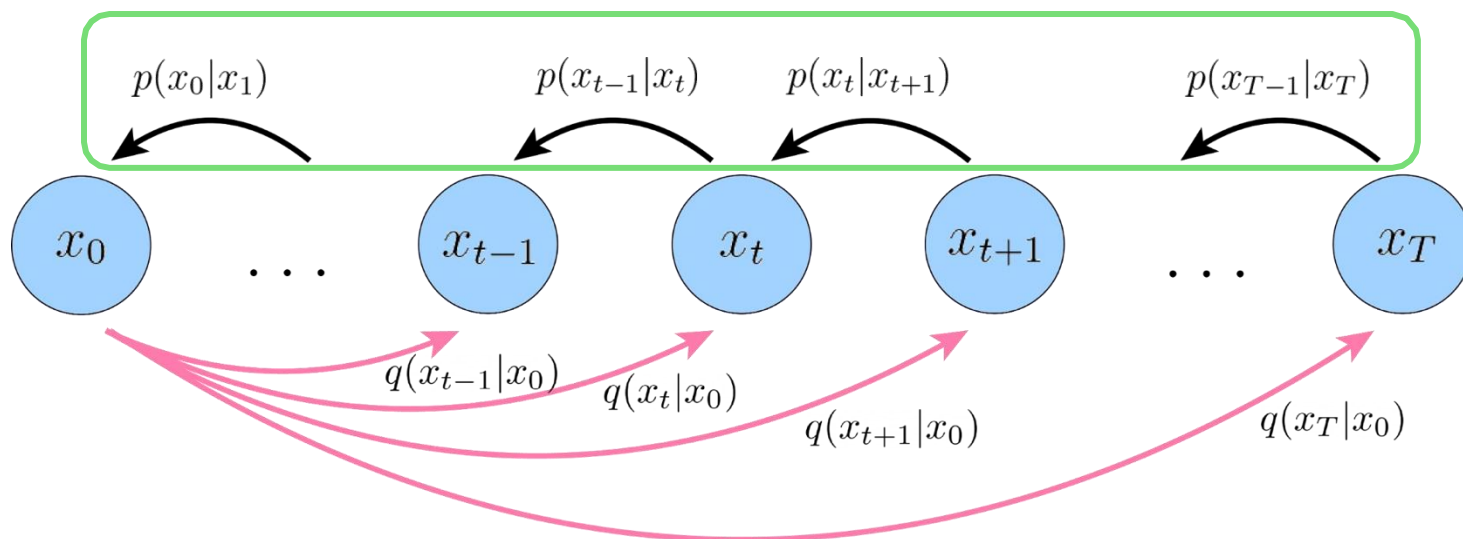


We learn to decode



Diffusion Models

- A diffusion model is implemented as a single neural network (the decoder)



...can we formulate this as supervised learning?

Optimization

- We want to learn a denoising decoder:



$$\hat{x}_{t-1} = \mu_{\text{dec}} + \sigma_{\text{dec}} * \epsilon$$

- Recall that:

$$q(x_{t-1}|x_0) = \mathcal{N}(x_{t-1}|x_0, \alpha_{t-1}^2 \mathbf{I})$$
$$\therefore x_{t-1} = x_0 + \alpha_{t-1} * \epsilon$$

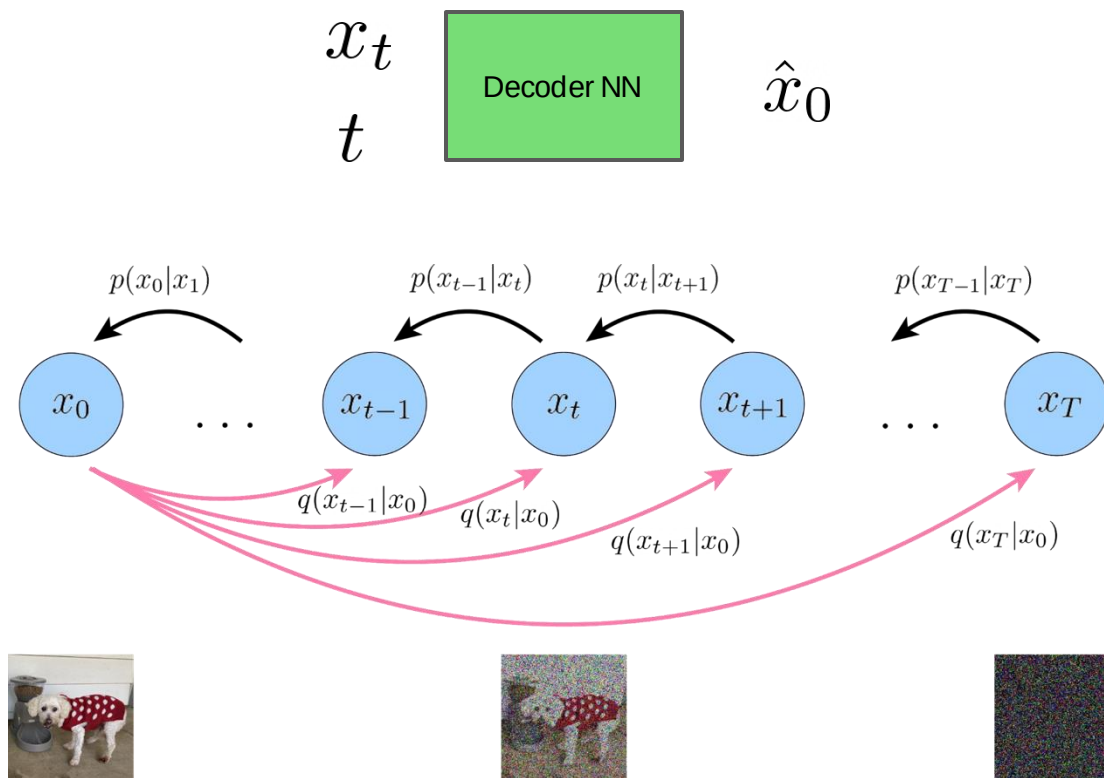
reparam. trick!

$$\epsilon \sim \mathcal{N}(0, \mathbf{I})$$

So, in the end, a diffusion model is simply *one* Neural Network that predicts a clean image x_0 from arbitrary noised image x_t

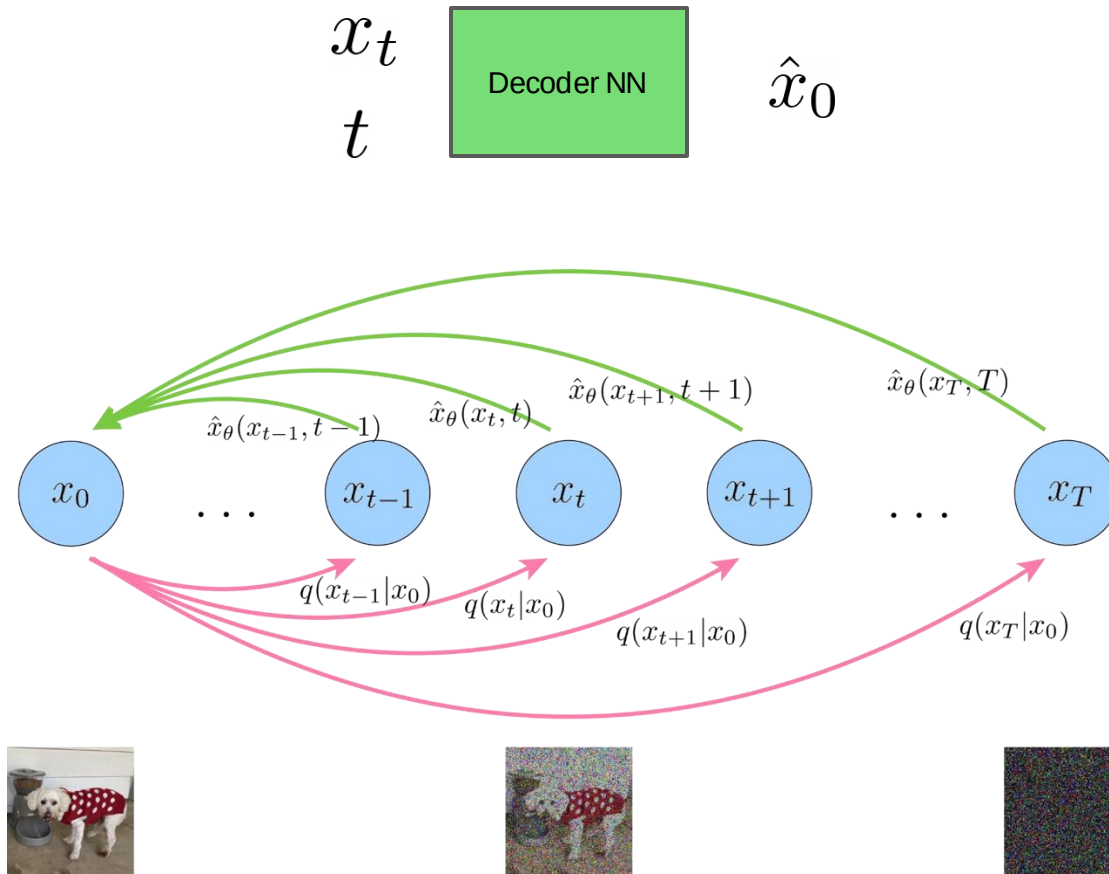
Diffusion Models: A Summary

- One NN that predicts a clean image from a noisy version of the image

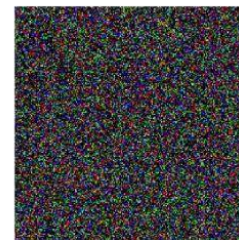
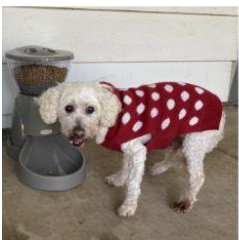
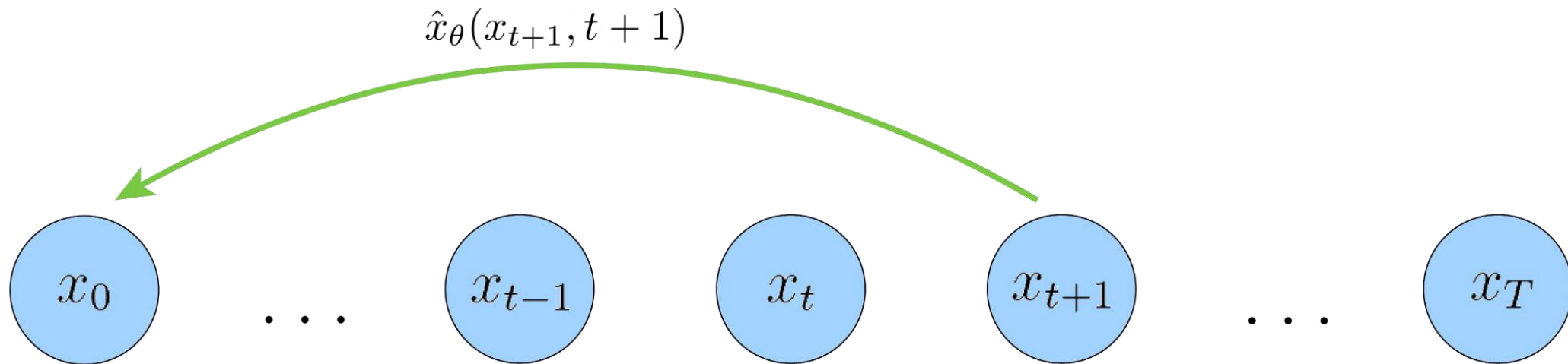


Diffusion Models: A Summary

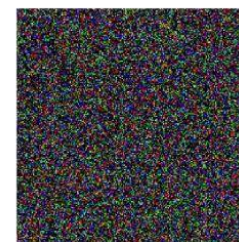
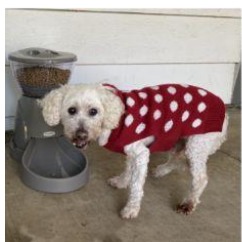
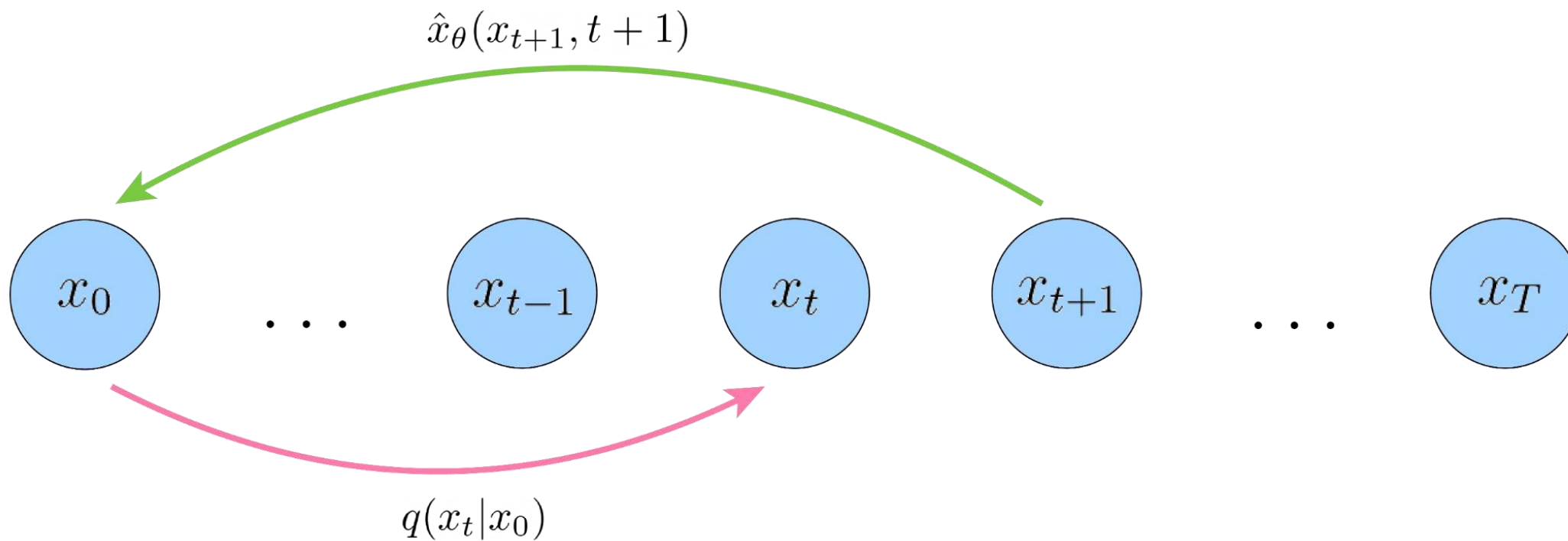
- One NN that predicts a clean image from a noisy version of the image



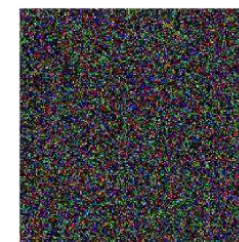
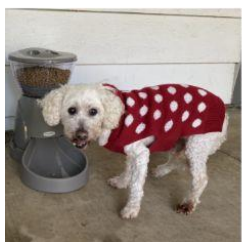
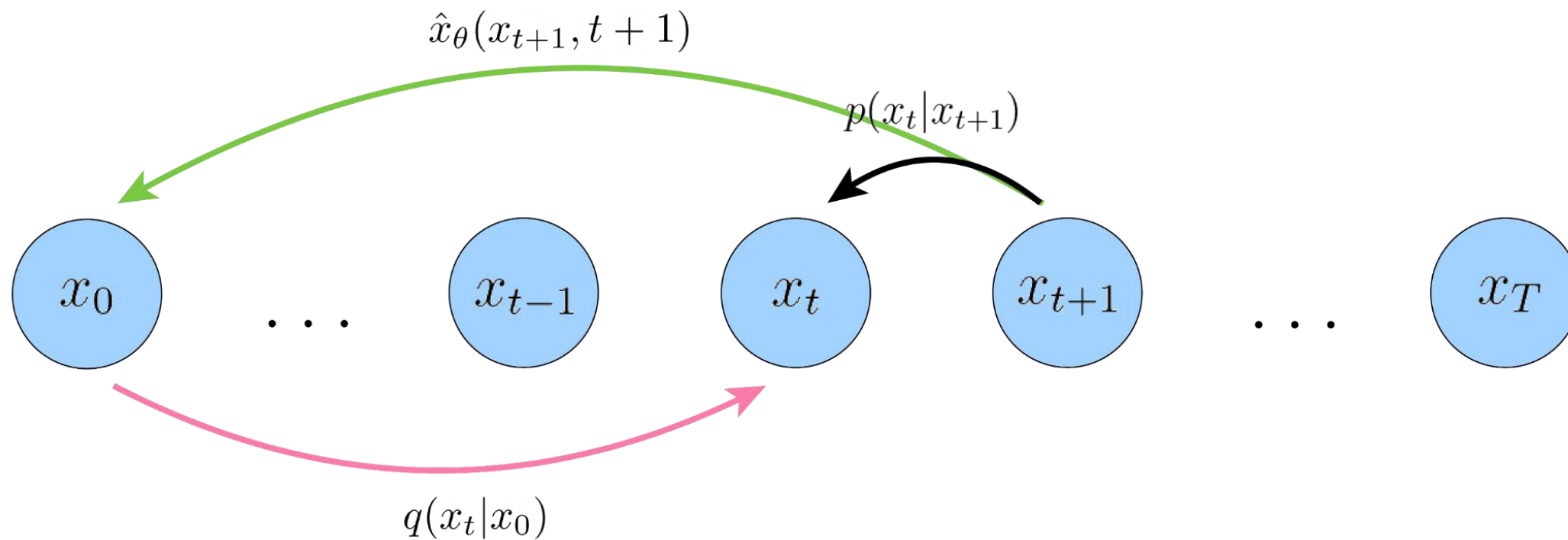
Sampling



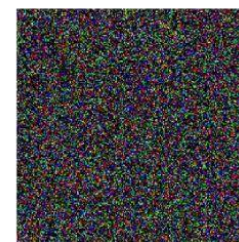
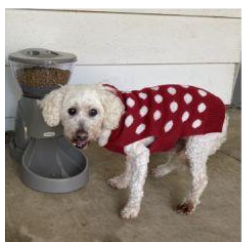
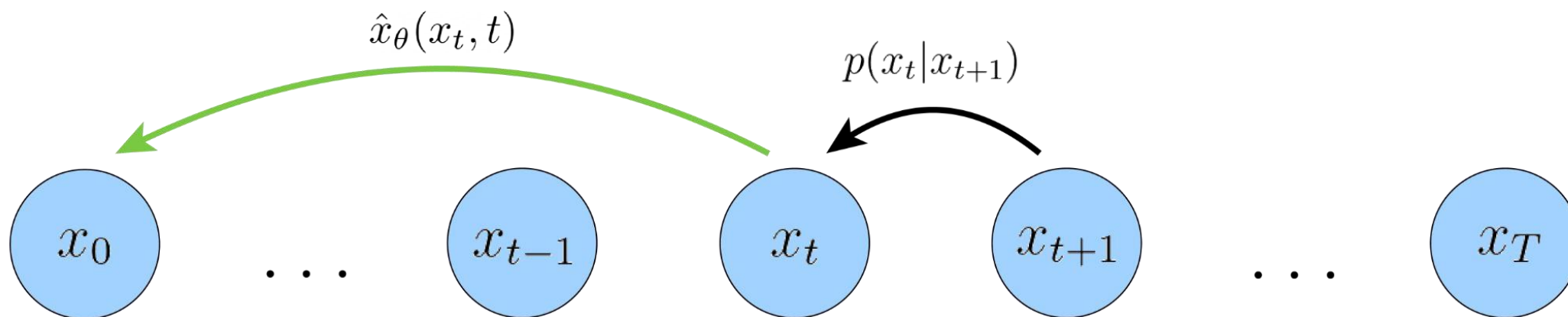
Sampling



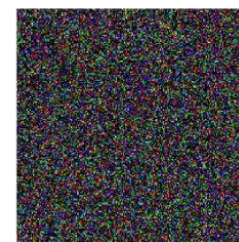
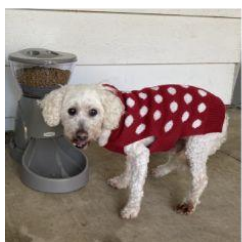
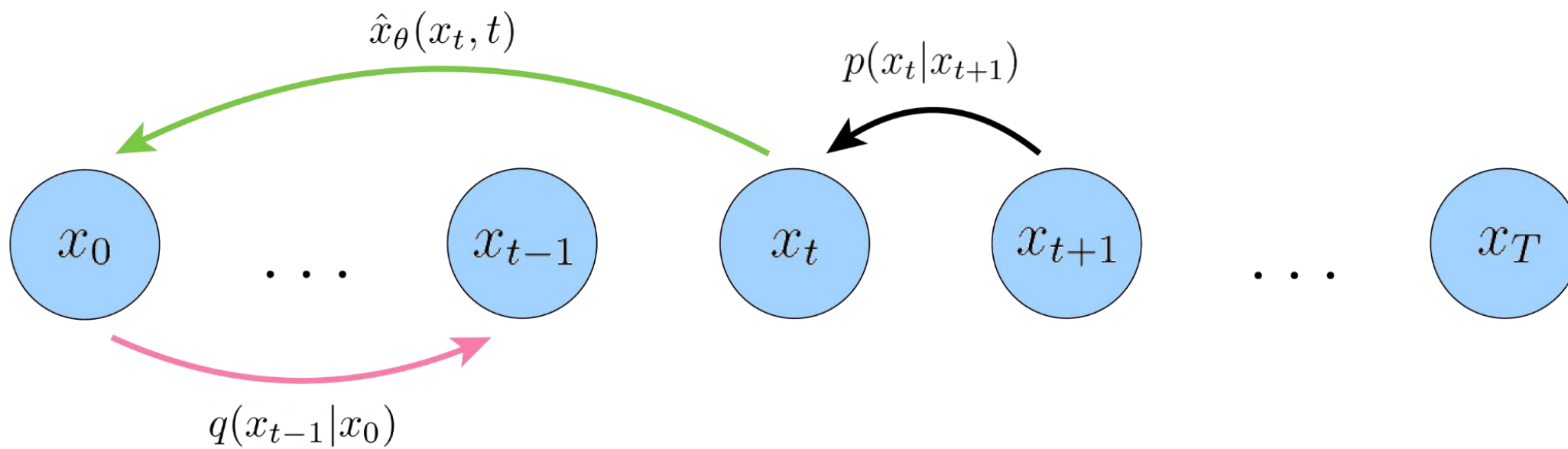
Sampling



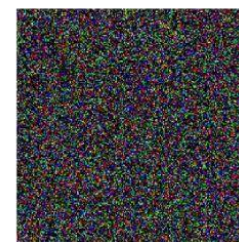
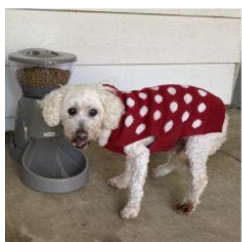
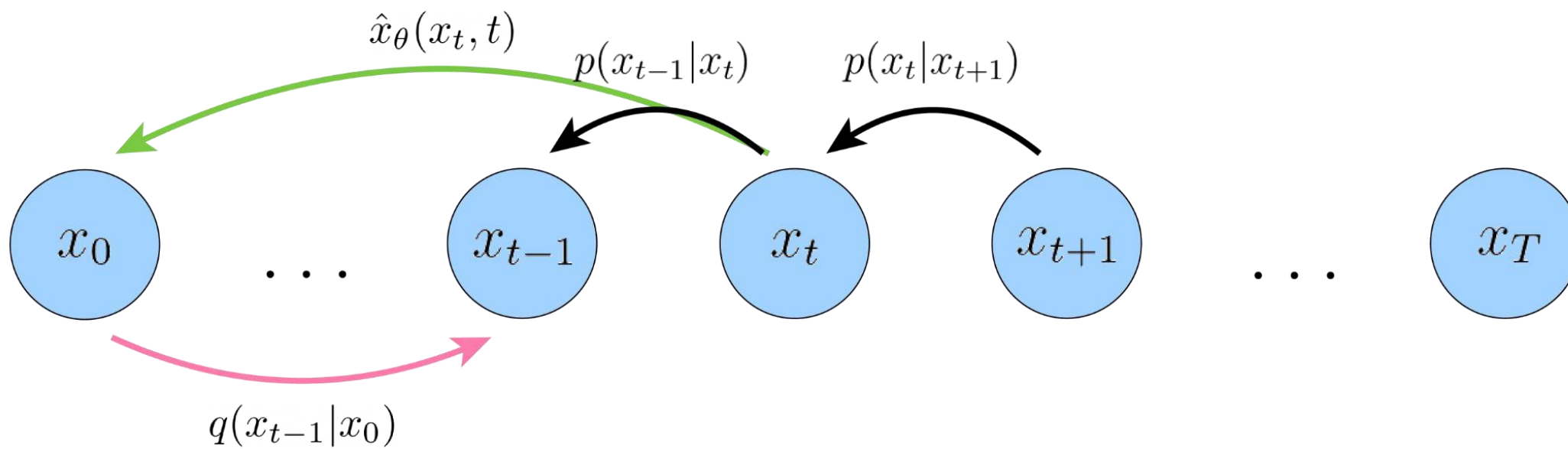
Sampling



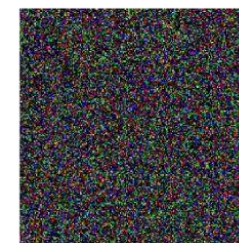
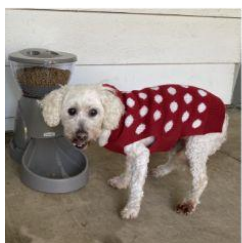
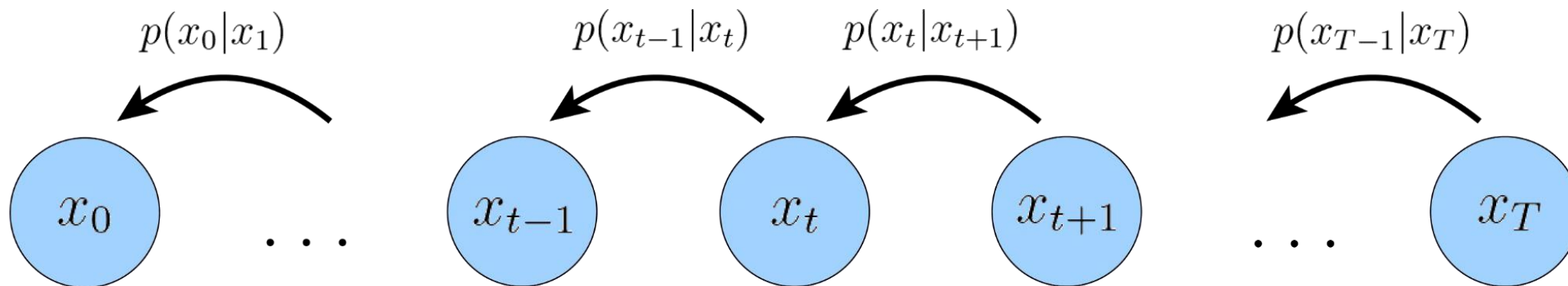
Sampling



Sampling



Sampling



Pseudocode

Algorithm 1 Training

```
1: repeat
2:    $\mathbf{x}_0 \sim q(\mathbf{x}_0)$  Sample image
3:    $t \sim \text{Uniform}(1, \dots, T)$  Sample step
4:    $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$  Sample noise
5:   Take gradient descent step on
6:      $\nabla_{\theta} \|\mathbf{x}_0 - \hat{\mathbf{x}}_{\theta}(\mathbf{x}_0 + \alpha_t \epsilon, t)\|^2$ 
7: until converged
```

Algorithm 2 Sampling

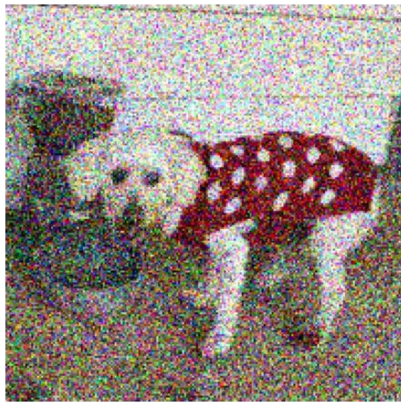
```
1:  $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
2: for  $t = T, \dots, 1$ :
3:    $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$  if  $t > 1$ , else  $\epsilon = 0$ 
4:    $\mathbf{x}_{t-1} = \hat{\mathbf{x}}_{\theta}(\mathbf{x}_t, t) + \alpha_{t-1} \epsilon$ 
5: end for Predict the initial image
6: return  $\mathbf{x}_0$ 
```

Three Different Interpretations

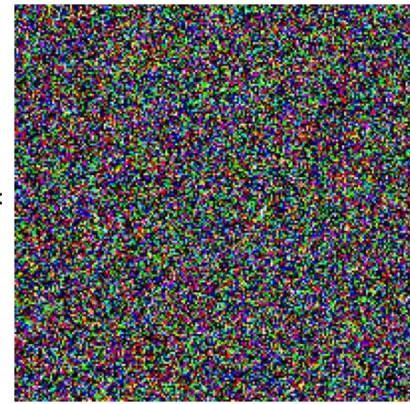
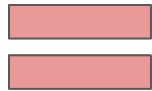
- Training diffusion models can be done using three different interpretations:
 - Predicting original image  (we just did this)
 - Predicting the noise 
 - Predicting a score function 100

Image 🖼️ and Noise 🔊 relation

- Recall that our objective is to predict $\hat{x}_\theta(x_t, t) \approx x_0$
- For arbitrary $x_t \sim q(x_t | x_0)$, we can rewrite it as $x_t = x_0 + \alpha_t \epsilon_0$
- Thus, predicting ϵ_0 determines x_0 and vice-versa



$$x_t \sim q(x_t | x_0)$$



Noise scheduler

- Re-applying the formula leads to the following:

$$q(x_t|x_0) = \mathcal{N}(x_t, \sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t) I)$$

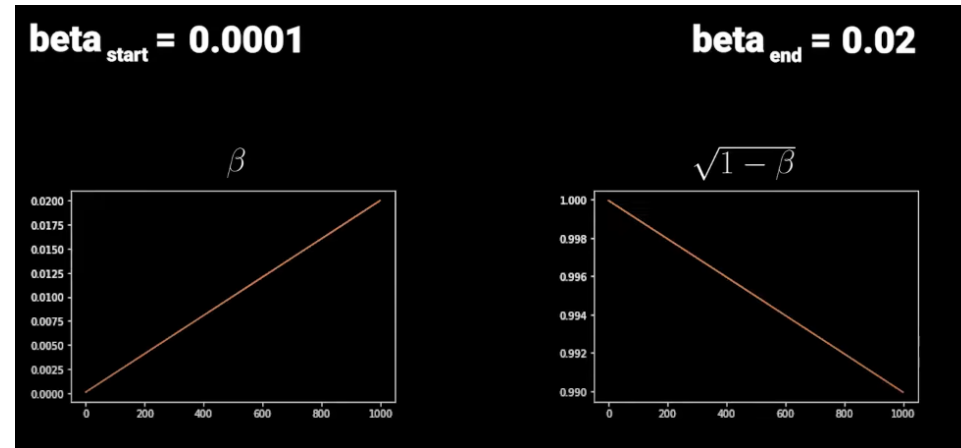
Where $\alpha_t = 1 - \beta_t$, $\bar{\alpha}_t = \prod_{i=1}^t \alpha_i$,

Linear noise schedule

- Add noise:

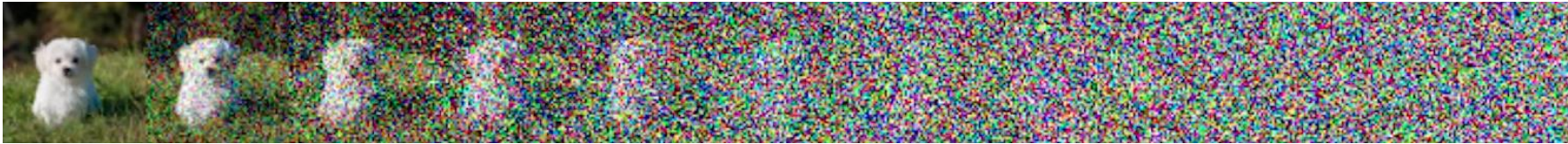
$$q(x_t|x_{t-1}) = \mathcal{N}\left(x_t, \underset{\substack{\uparrow \\ \text{mean}}}{\sqrt{1 - \beta_t}}x_{t-1}, \underset{\substack{\downarrow \\ \text{variance}}}{\sqrt{\beta_t}I}\right)$$

- The β_t is defined by a **schedule**, e.g., linear schedule:

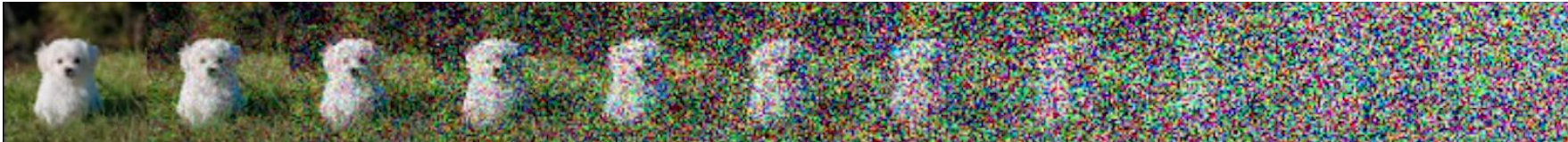


Different Schedule

- Add noise gradually, regulated by a **schedule**:
 - Linear Schedule⁽¹⁾



- Cosine Schedule⁽²⁾

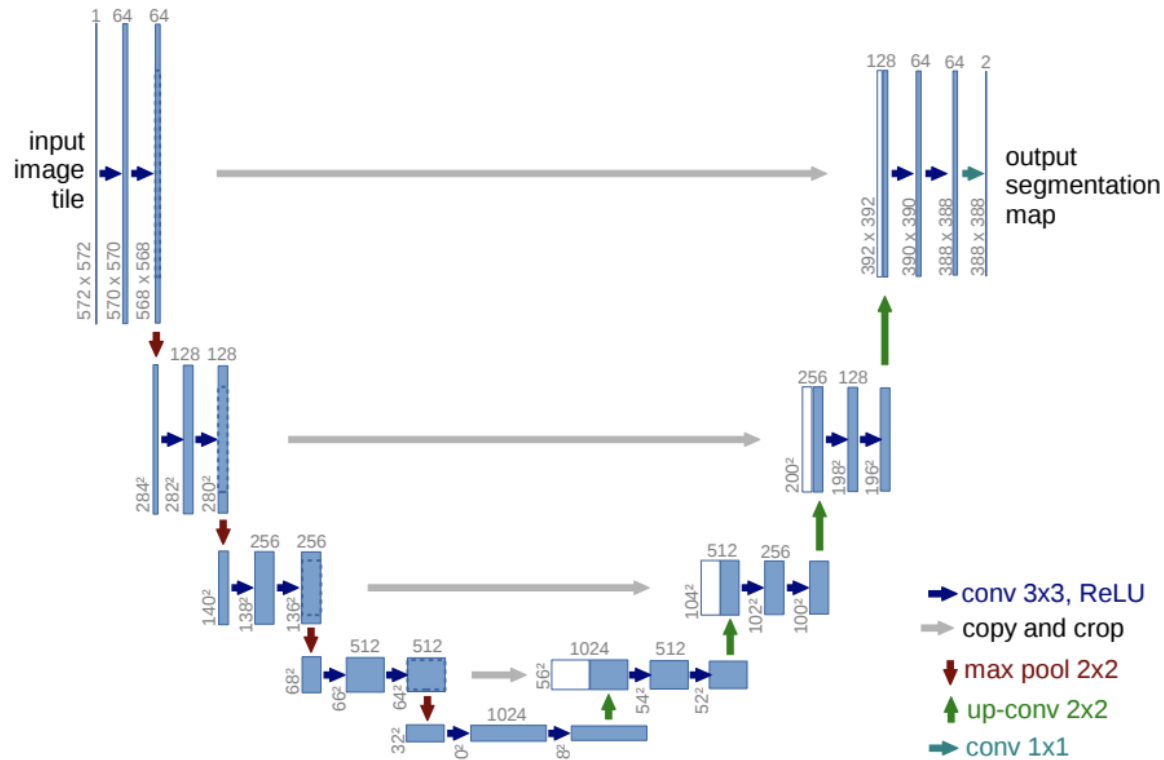


See <https://huggingface.co/docs/diffusers/en/using-diffusers/schedulers>

(1) - Denoising Diffusion Probabilistic Models (Ho et al. 2020)

(2) - Improved Denoising Diffusion Probabilistic Models (Nichol et al. 2020)

Predict the noise with U-Net (+Improvements)



Improvements:

- Attention
- Group norm
- ConvNext
- BigGAN ResNet Blocks
- Swish/GeLU
- etc

Fig. 1. U-net architecture (example for 32x32 pixels in the lowest resolution). Each blue box corresponds to a multi-channel feature map. The number of channels is denoted on top of the box. The x-y-size is provided at the lower left edge of the box. White boxes represent copied feature maps. The arrows denote the different operations.

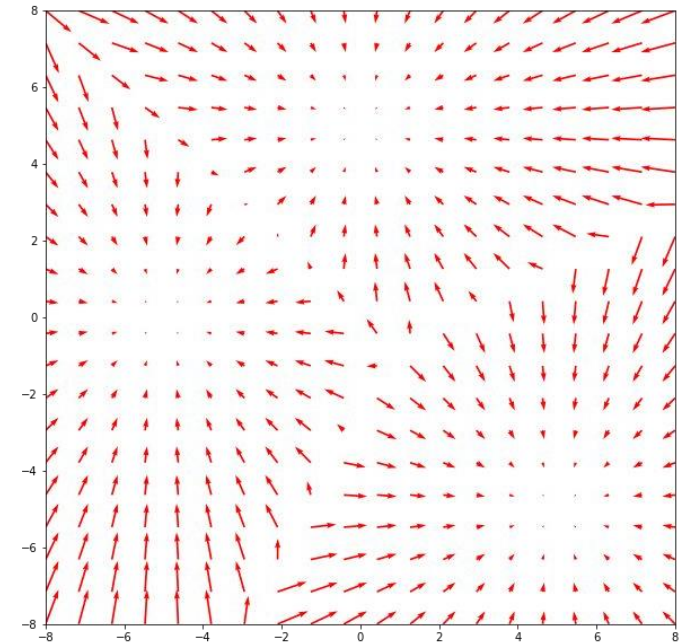
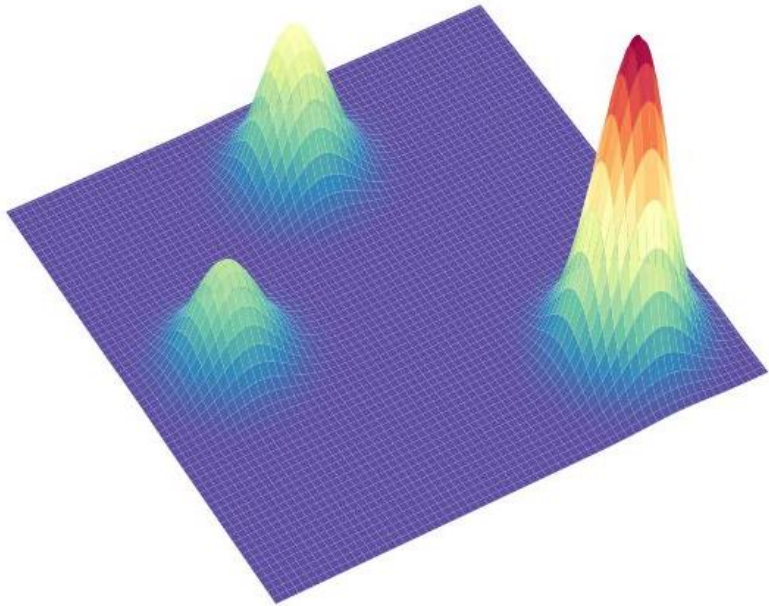
See implementation at: <https://nn.labml.ai/diffusion/ddpm/unet.html>

Score Functions 100

- What are score functions?

$$\nabla_x \log p(\mathbf{x})$$

- Intuitively, they describe how to move in data space to improve the (log) likelihood.



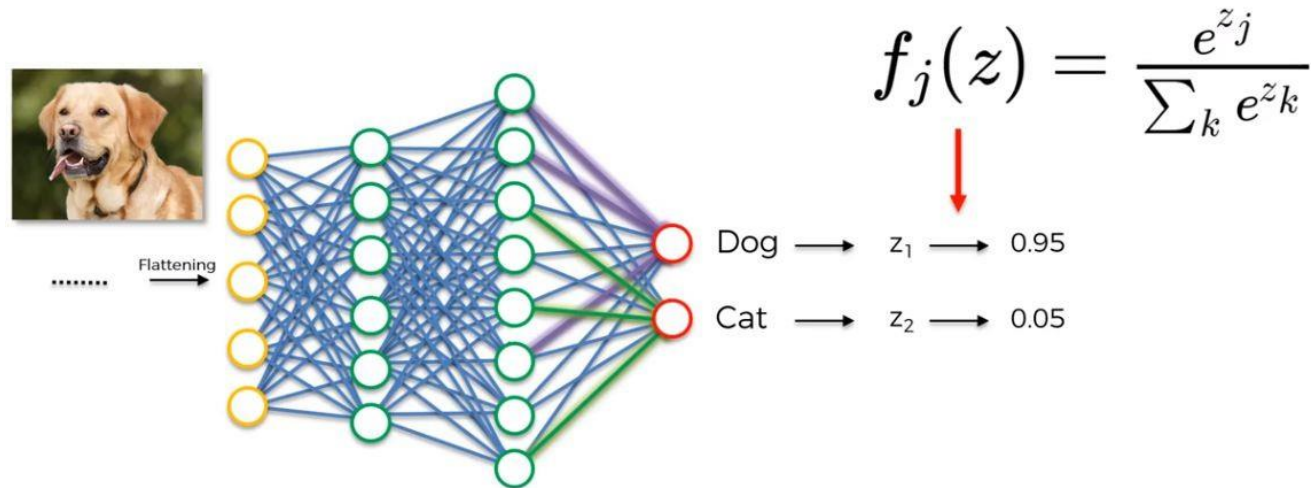
Probability-based Generative Modeling

- Why is modeling the **probability** hard in generative models?
- The model output need to be
 - Non-negative: $0 \leq p_{\theta}(\mathbf{x})$
 - Less than or equal to 1: $p_{\theta}(\mathbf{x}) \leq 1$
 - Sum to 1 for the **entire** space: $\int_{\mathbf{x}} p_{\theta}(\mathbf{x}) d\mathbf{x} = 1$

Impossible to model the **probability of all possible**

Probability-based Discriminative Modeling

- We have a **pre-defined** list of **all** possible outputs (labels)



A simple observation...

- All probability distributions can be written as:

$$p_{\theta}(\mathbf{x}) = \frac{1}{Z_{\theta}} e^{-f_{\theta}(\mathbf{x})}$$

- the unconstrained $f_{\theta}(\mathbf{x})$ value called the **energy**.

- Z_{θ} is a normalization constant, computed as: $Z_{\theta} = \int_{\mathbf{x}} e^{-f_{\theta}(\mathbf{x})} d\mathbf{x}$

Energy-based Generative Modeling ⚡

- We can model the energy function $f_{\theta}(\mathbf{x})$ using a neural network (namely *energy-based model (EBM)*)
- How do we train our energy function?
 - We can try interpreting it as a probability: $p_{\theta}(\mathbf{x}) = \frac{1}{Z_{\theta}} e^{-f_{\theta}(\mathbf{x})}$
 - Then we can maximize log likelihood as before: $\max_{\theta} \sum_{i=1}^N \log p_{\theta}(\mathbf{x}_i)$
 - What is the problem with this?

$$Z_{\theta} = \int_{\mathbf{x}} e^{-f_{\theta}(\mathbf{x})} d\mathbf{x}$$

Intractable for complex parameterizations!

How to avoid calculating Z_θ ?

- Remember that Z_θ is a *constant* that only depends on parameters θ
- Then, if we take the input gradient of the log of the probability:

$$\begin{aligned}\nabla_{\mathbf{x}} \log p_\theta(\mathbf{x}) &= \nabla_{\mathbf{x}} \log\left(\frac{1}{Z_\theta} e^{-f_\theta(\mathbf{x})}\right) \\ &= \nabla_{\mathbf{x}} \log \frac{1}{Z_\theta} + \nabla_{\mathbf{x}} \log e^{-f_\theta(\mathbf{x})} \\ &= -\nabla_{\mathbf{x}} f_\theta(\mathbf{x}) \\ &\approx \mathbf{s}_\theta(\mathbf{x})\end{aligned}$$

- The $\nabla_{\mathbf{x}} \log p_\theta(\mathbf{x})$ is called the **score function**.

How do we train our score function?

- Minimize distance between our score model and the ground truth score:

$$\mathbb{E}_{p(\mathbf{x})} \left[\|\nabla \log p(\mathbf{x}) - \mathbf{s}_{\theta}(\mathbf{x})\|_2^2 \right]$$

- We don't have access to $\nabla_x \log p_{\theta}(x)$ but there are ways to minimize this objective without knowledge of the ground-truth data score [1,2,3]

1.Hyvarinen, A. (2005). Estimation of non-normalized statistical models by score matching. *Journal of Machine Learning Research*, 6(Apr), 695-709.

2.Vincent, P. (2011). A connection between score matching and denoising autoencoders. *Neural Computation*, 23(7), 1661-1674. MIT Press.

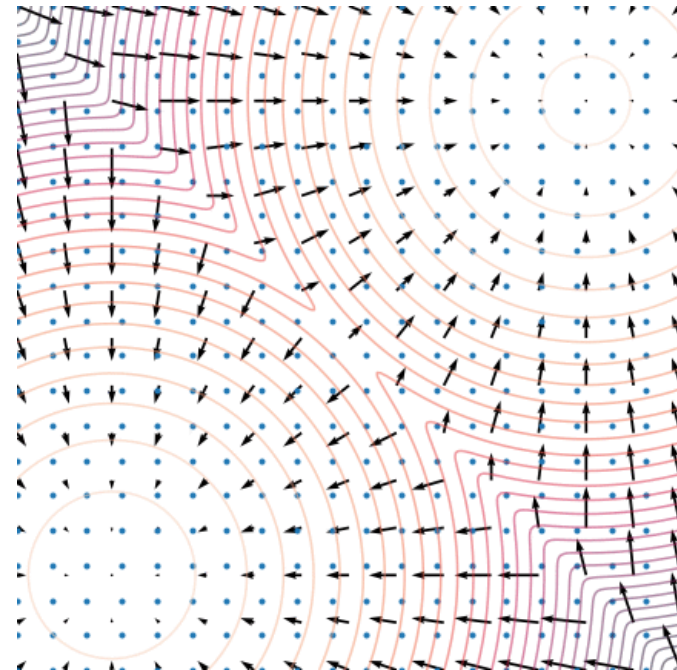
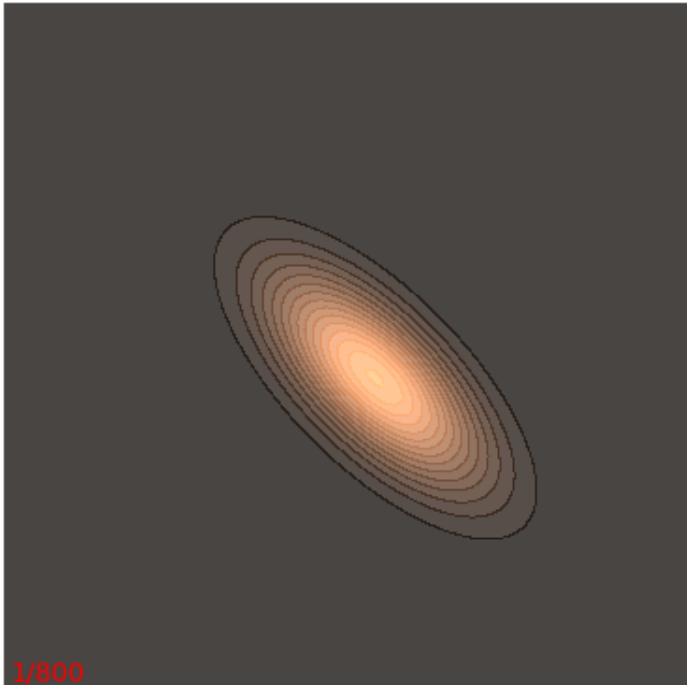
3.Song, Y., Garg, S., Shi, J., & Ermon, S. (2020). Sliced score matching: A scalable approach to density and score estimation. *Uncertainty in Artificial Intelligence*, 574-584.

Using Langevin dynamics to sample

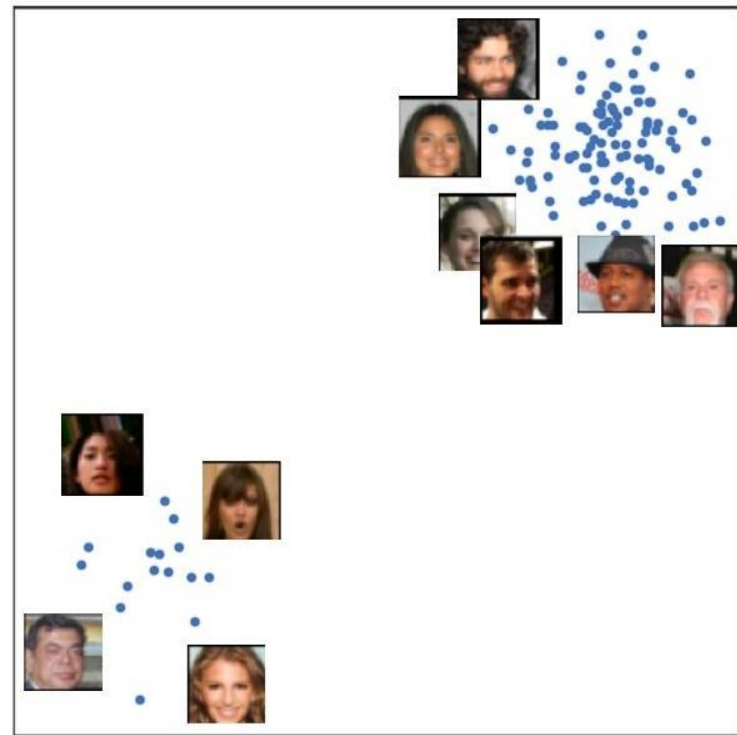
- Once we have trained a score-based model, we can use Langevin dynamics:

$$\mathbf{x}_{i+1} \leftarrow \mathbf{x}_i + c \nabla_{\mathbf{x}} \log p(\mathbf{x}) + \sqrt{2c} \boldsymbol{\epsilon}_i, \quad i = 0, 1, \dots, K,$$

Add randomness to explore multiple modes



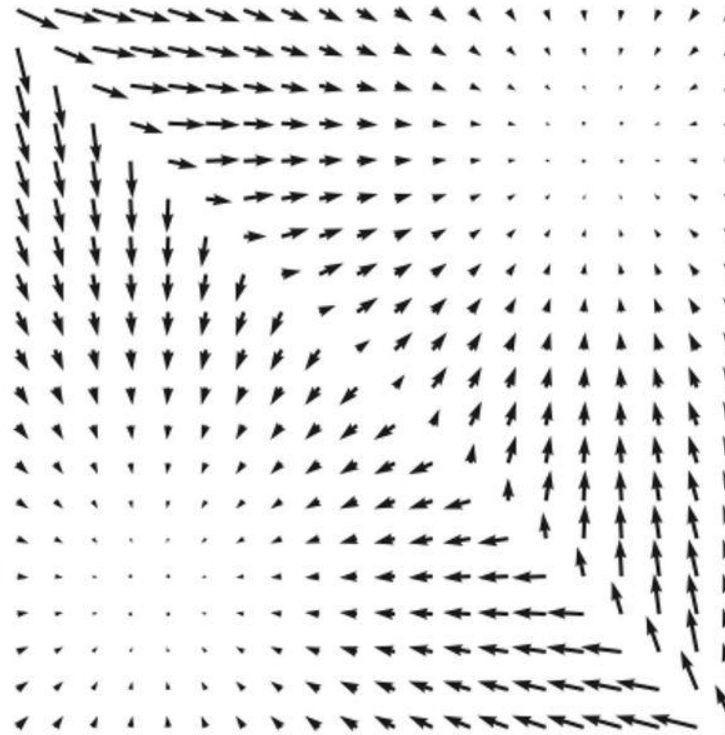
Score-based Generative Modeling 100



Data samples

$$\{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N\} \stackrel{\text{i.i.d.}}{\sim} p(\mathbf{x})$$

score
matching



Scores

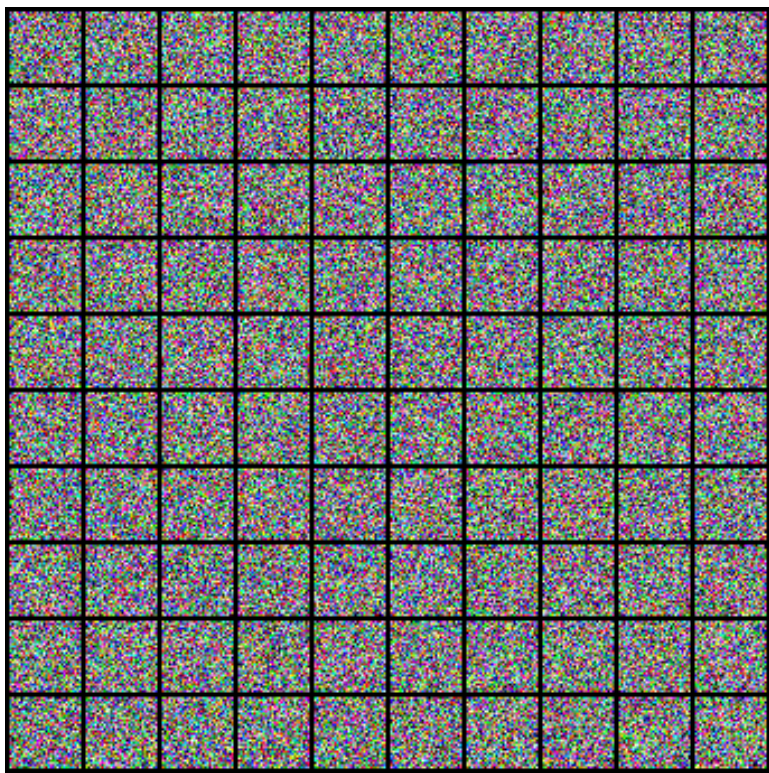
$$\mathbf{s}_\theta(\mathbf{x}) \approx \nabla_{\mathbf{x}} \log p(\mathbf{x})$$

Langevin
dynamics

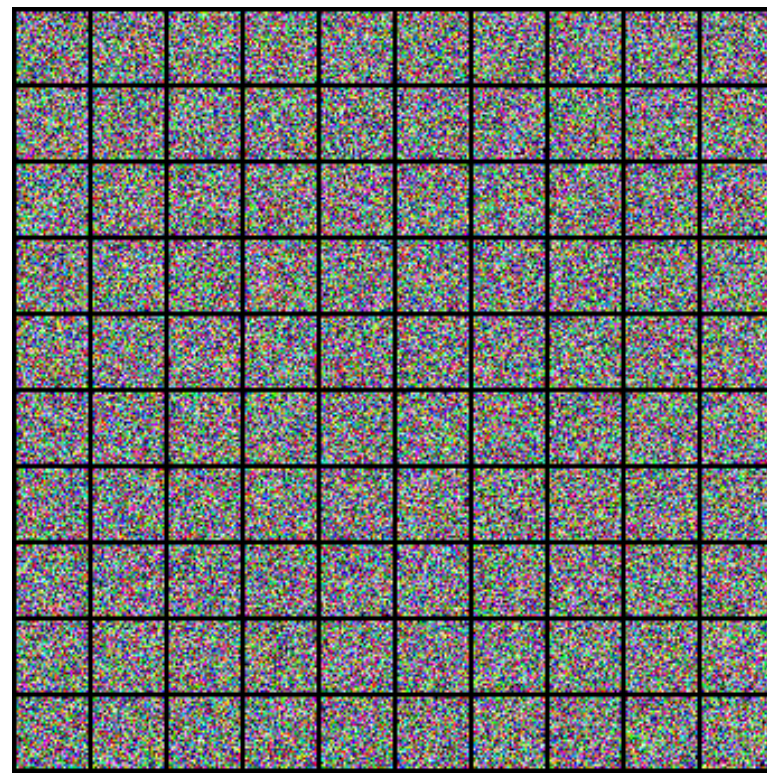


New samples

Results (resemble diffusion...)



CelebA



CIFAR-10



Score 100 and Noise 📢 ?

- Tweedie's Formula:

$$\mathbf{x}_0 \approx \mathbf{x}_t + \alpha_t^2 \nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t)$$

- We can derive by equating Tweedie's formula with the Reparameterization Trick:

$$\mathbf{x}_0 = \mathbf{x}_t + \alpha_t^2 \nabla \log p(\mathbf{x}_t) = \mathbf{x}_t - \alpha_t \boldsymbol{\epsilon}_0$$

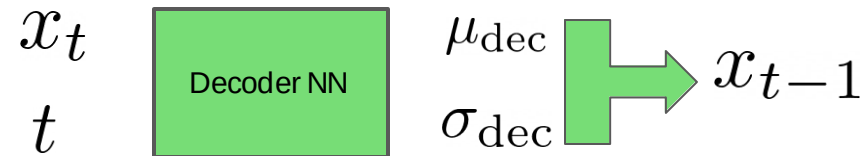
$$\therefore \alpha_t^2 \nabla \log p(\mathbf{x}_t) = -\alpha_t \boldsymbol{\epsilon}_0$$

$$\nabla \log p(\mathbf{x}_t) = -\frac{1}{\alpha_t} \boldsymbol{\epsilon}_0$$

- Intuitively, the direction to move in data space towards a natural image is the negative noise term that was added.

Summary : Variational Diffusion Models

- We want to learn a denoising decoder:



$$\hat{x}_{t-1} = \mu_{\text{dec}} + \sigma_{\text{dec}} * \epsilon$$

- Recall that:

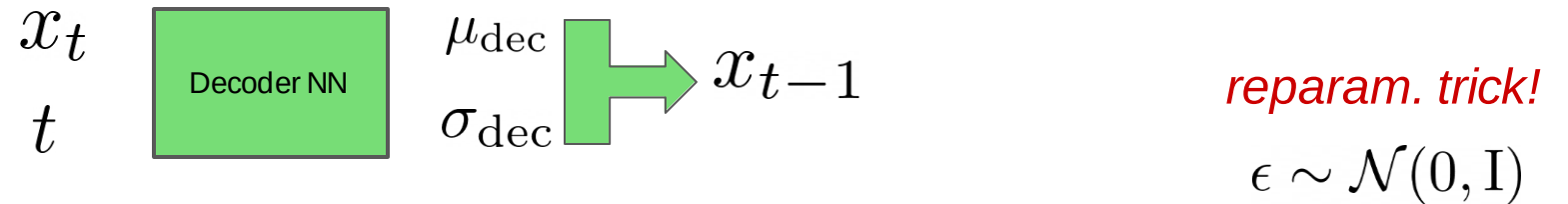
$$q(x_{t-1}|x_0) = \mathcal{N}(x_{t-1}|x_0, \alpha_{t-1}^2 \mathbf{I})$$
$$\therefore x_{t-1} = x_0 + \alpha_{t-1} * \epsilon$$

reparam. trick!

$$\epsilon \sim \mathcal{N}(0, \mathbf{I})$$

Summary: Variational Diffusion Models

- We want to learn a denoising decoder:

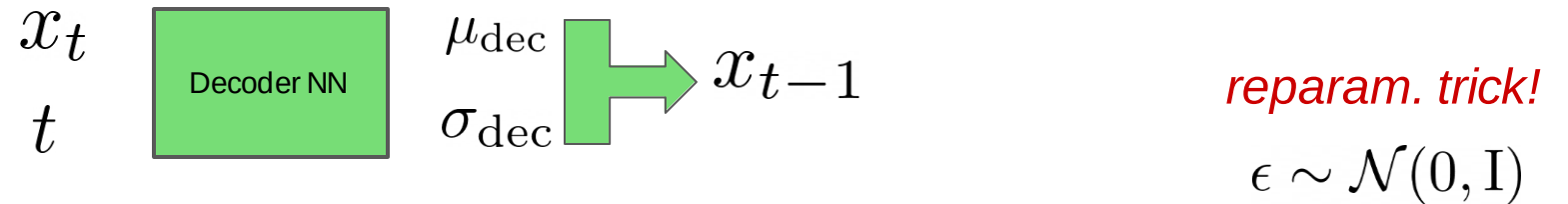


- Where ground truth denoising sample is: $x_{t-1} = x_0 + \alpha_{t-1} \cdot \epsilon$
- Loss Objective:

$$\arg \min_{\theta} \sum_{t=1}^T \|x_0 - \hat{x}_{\theta}(x_t, t)\|_2^2$$

Summary : Variational Diffusion Models

- We want to learn a denoising decoder:

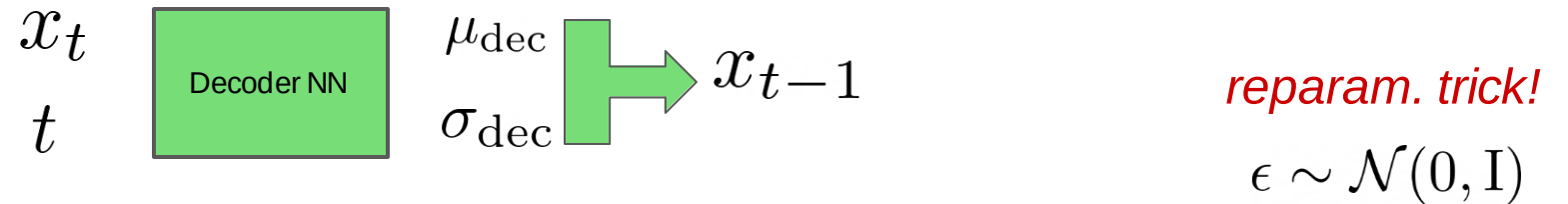


- Where ground truth denoising sample is: $x_{t-1} = x_0 + \alpha_{t-1} \cdot \epsilon$
- Which is equivalent to: $x_{t-1} = (x_t - \alpha_t \cdot \epsilon_0) + \alpha_{t-1} \cdot \epsilon$ (noise prediction)
- Loss Objective:

$$\arg \min_{\theta} \sum_{t=1}^T \|\epsilon_0 - \hat{\epsilon}_{\theta}(x_t, t)\|_2^2$$

Summary : Variational Diffusion Models

- We want to learn a denoising decoder:



- Where ground truth denoising sample is: $x_{t-1} = x_0 + \alpha_{t-1} \cdot \epsilon$
- Which is equivalent to: $x_{t-1} = (x_t - \alpha_t \cdot \epsilon_0) + \alpha_{t-1} \cdot \epsilon$
- Which is equivalent to: $x_{t-1} = \left(x_t + \alpha_t^2 \cdot \nabla_{x_t} \log p(x_t) \right) + \alpha_{t-1} \cdot \epsilon$
(score prediction)

- Loss Objective:

$$\arg \min_{\theta} \sum_{t=1}^T \|\nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t) - \hat{\mathbf{s}}_{\theta}(\mathbf{x}_t, t)\|_2^2$$

Conditional Diffusion Models

- How do we incorporate conditional information, to control data generation?

“a painting of a fox sitting in a field at sunrise in the style of Claude Monet”



ImageN



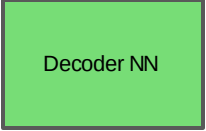
StableDiffusion

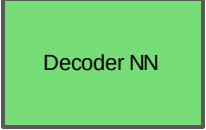


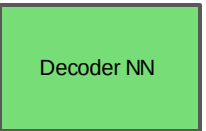
Dall-E 2.0

Implemented as a neural net that

-  Predicts original image $\hat{x}_{\theta}(\mathbf{x}_t, t) \approx \mathbf{x}_0 \longrightarrow \hat{x}_{\theta}(\mathbf{x}_t, t, y) \approx \mathbf{x}_0$

$\begin{matrix} \mathbf{x}_t \\ t \\ y \end{matrix}$

 $\hat{\mathbf{x}}_0$
-  Predicts noise epsilon $\hat{\epsilon}_{\theta}(\mathbf{x}_t, t) \approx \epsilon_0 \longrightarrow \hat{\epsilon}_{\theta}(\mathbf{x}_t, t, y) \approx \epsilon_0$

$\begin{matrix} \mathbf{x}_t \\ t \\ y \end{matrix}$

 $\hat{\epsilon}_0$
- 100 Predicts score function $s_{\theta}(\mathbf{x}_t, t) \approx \nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t) \longrightarrow s_{\theta}(\mathbf{x}_t, t, y) \approx \nabla \log p(\mathbf{x}_t \mid y)$

$\begin{matrix} \mathbf{x}_t \\ t \\ y \end{matrix}$

 $\nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t \mid y)$

Hard to calibrate the conditioning strength

Classifier Guidance

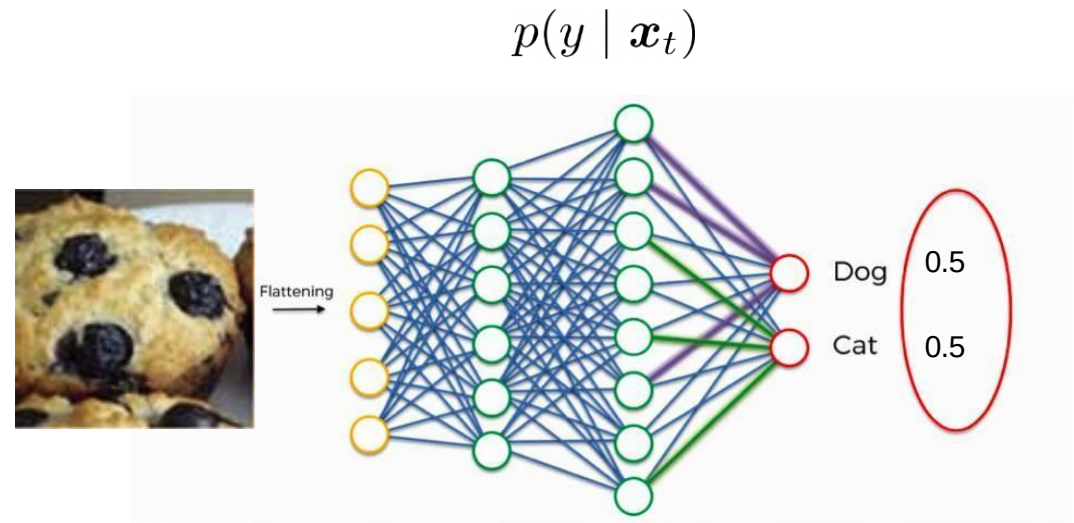
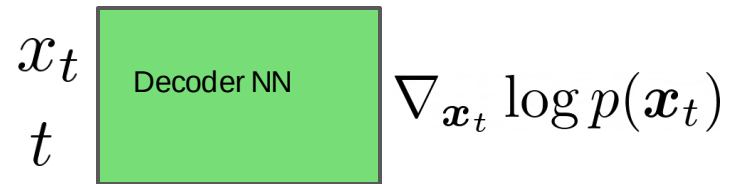
- Consider the scoring interpretation; we are interested in learning $\nabla \log p(x_t | y)$ rather than score function $\nabla \log p(x_t)$
- From Bayes' theorem:

$$\begin{aligned}\nabla \log p(\mathbf{x}_t | y) &= \nabla \log \left(\frac{p(\mathbf{x}_t)p(y | \mathbf{x}_t)}{p(y)} \right) \\ &= \nabla \log p(\mathbf{x}_t) + \nabla \log p(y | \mathbf{x}_t) - \nabla \log p(y) \\ &= \underbrace{\nabla \log p(\mathbf{x}_t)}_{\text{unconditional score}} + \underbrace{\nabla \log p(y | \mathbf{x}_t)}_{\text{adversarial gradient}} \\ &\quad \text{(Classifier)}\end{aligned}$$

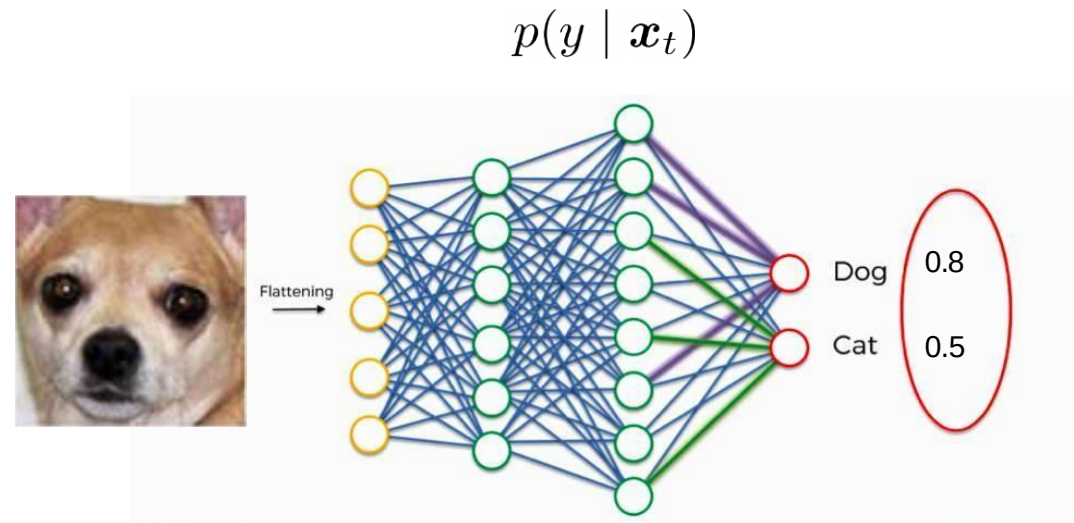
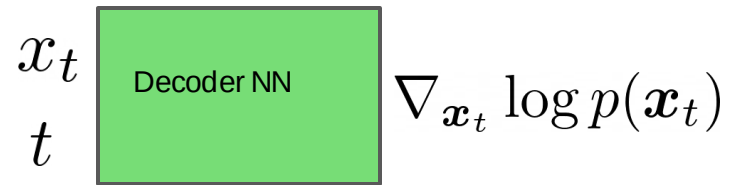


$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

Sampling w\ classifier guidance

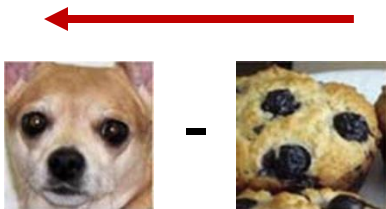


Sampling w\ classifier guidance



Sampling w\ classifier guidance

- We approximate the following:

$$\nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t) + \left(\text{dog} - \text{cookie} \right)$$


- I.e.,

$$\nabla \log p(\mathbf{x}_t \mid y) = \nabla \log p(\mathbf{x}_t) + \gamma \nabla \log p(y \mid \mathbf{x}_t)$$

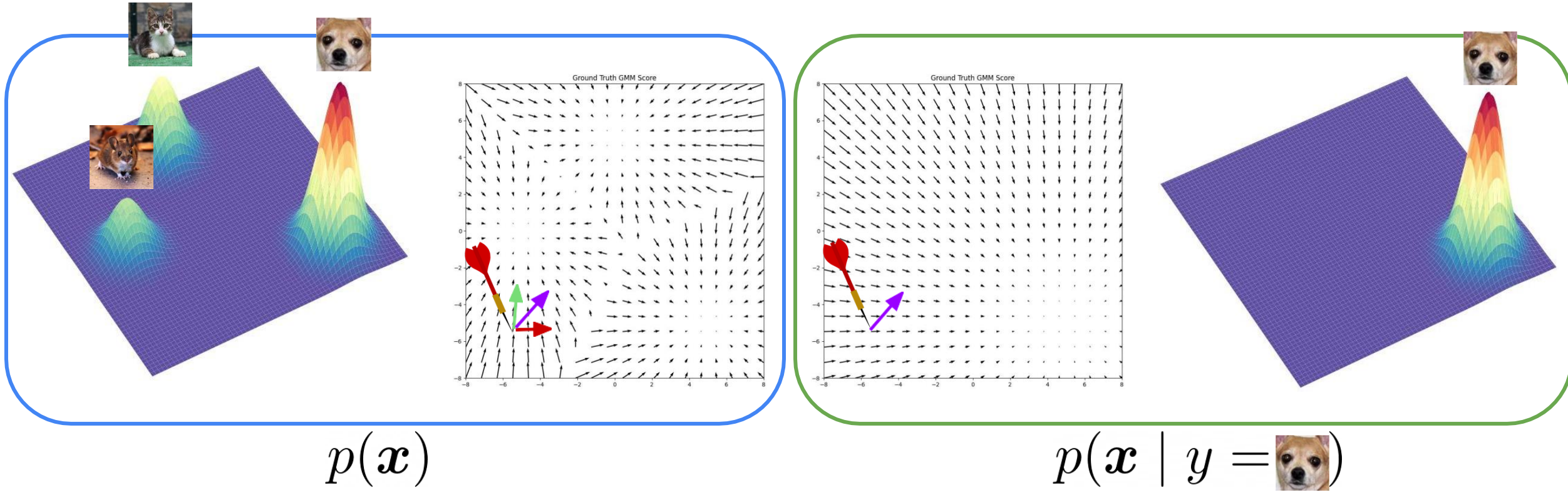
Guidance scale

Noised image

Classifier Guidance

- Sampling is then done by querying the learned unconditional score function as well as the adversarial gradient of a classifier.

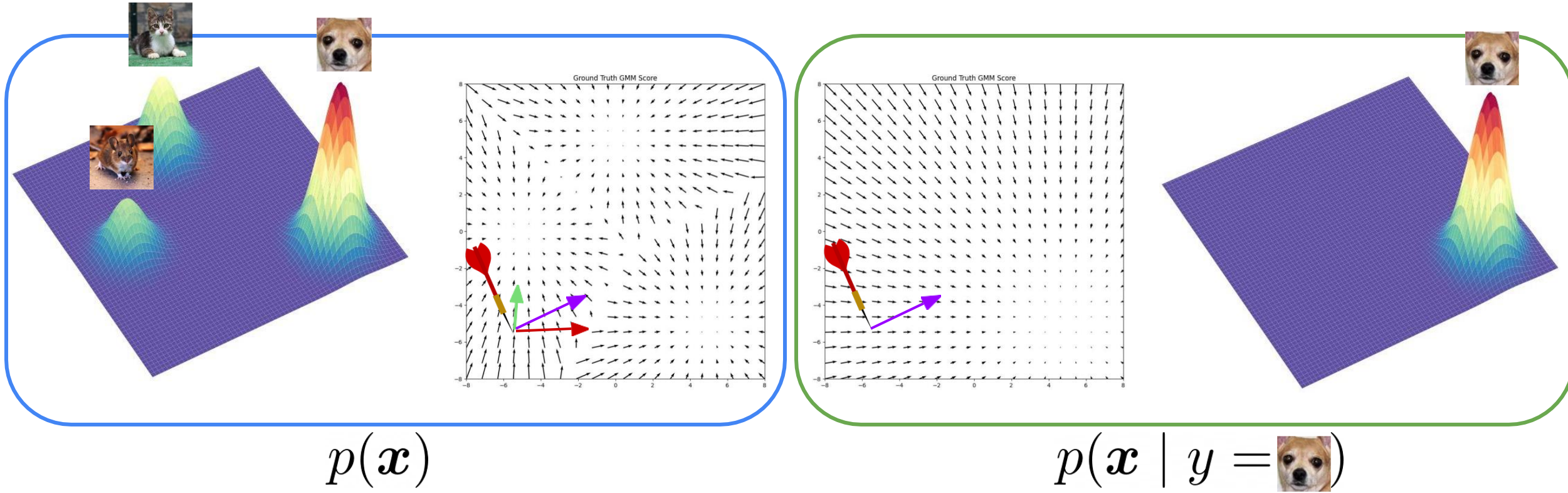
$$\nabla \log p(\mathbf{x}_t \mid y) = \nabla \log p(\mathbf{x}_t) + \gamma \nabla \log p(y \mid \mathbf{x}_t)$$



Classifier Guidance

- Sampling is then done by querying the learned unconditional score function as well as the adversarial gradient of a classifier.

$$\nabla \log p(\mathbf{x}_t \mid y) = \nabla \log p(\mathbf{x}_t) + \gamma \nabla \log p(y \mid \mathbf{x}_t)$$



Classifier-Free Guidance

Let's revisit the score of a conditional diffusion model:

$$\nabla \log p(\mathbf{x}_t \mid y) = \nabla \log p(y \mid \mathbf{x}_t) + \nabla \log p(\mathbf{x}_t)$$

Rearranging, we get:

$$\nabla \log p(y \mid \mathbf{x}_t) = \nabla \log p(\mathbf{x}_t \mid y) - \nabla \log p(\mathbf{x}_t)$$

Let's take the Classifier Guidance formulation we just derived:

$$\nabla \log p(\mathbf{x}_t \mid y) = \nabla \log p(\mathbf{x}_t) + \gamma \nabla \log p(y \mid \mathbf{x}_t)$$

Then, substituting in our rearrangement, we get:

$$\begin{aligned} \nabla \log p(\mathbf{x}_t \mid y) &= \nabla \log p(\mathbf{x}_t) + \gamma (\nabla \log p(\mathbf{x}_t \mid y) - \nabla \log p(\mathbf{x}_t)) \\ &= \nabla \log p(\mathbf{x}_t) + \gamma \nabla \log p(\mathbf{x}_t \mid y) - \gamma \nabla \log p(\mathbf{x}_t) \\ &= \underbrace{\gamma \nabla \log p(\mathbf{x}_t \mid y)}_{\text{conditional score}} + \underbrace{(1 - \gamma) \nabla \log p(\mathbf{x}_t)}_{\text{unconditional score}} \end{aligned}$$

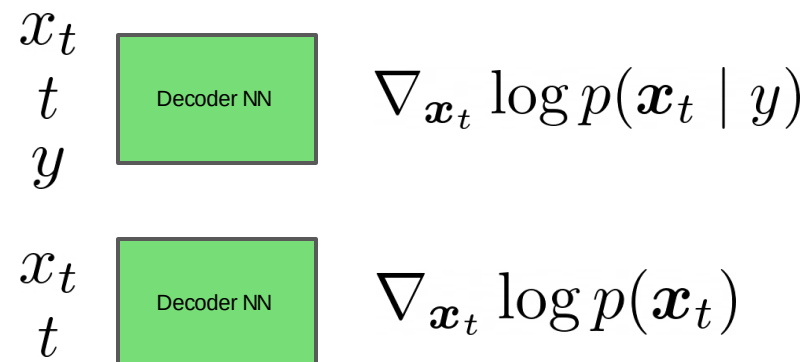
Classifier-Free Guidance

$$\nabla \log p(\mathbf{x}_t \mid y) = \underbrace{\gamma \nabla \log p(\mathbf{x}_t \mid y)}_{\text{conditional score}} + \underbrace{(1 - \gamma) \nabla \log p(\mathbf{x}_t)}_{\text{unconditional score}}$$

- What happens if γ is 1?
- What happens if γ is larger than 1?
- How many diffusion models do we need to train?

$$\mathbf{s}_\theta(\mathbf{x}_t, t, y) \approx \nabla \log p(\mathbf{x}_t \mid y)$$

$$\mathbf{s}_\theta(\mathbf{x}_t, t) \approx \nabla \log p(\mathbf{x}_t)$$



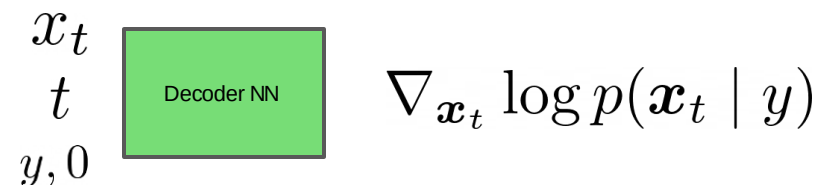
Classifier-Free Guidance

$$\nabla \log p(\mathbf{x}_t \mid y) = \underbrace{\gamma \nabla \log p(\mathbf{x}_t \mid y)}_{\text{conditional score}} + \underbrace{(1 - \gamma) \nabla \log p(\mathbf{x}_t)}_{\text{unconditional score}}$$

- What happens if γ is 1?
- What happens if γ is larger than 1?
- How many diffusion models do we need to train?

$$\mathbf{s}_\theta(\mathbf{x}_t, t, y) \approx \nabla \log p(\mathbf{x}_t \mid y)$$

$$\mathbf{s}_\theta(\mathbf{x}_t, t) \approx \nabla \log p(\mathbf{x}_t)$$



Just one - as long as we train it with dropout on the conditioning info!

A stained glass window of a panda eating bamboo.



Unconditioned

Conditioned

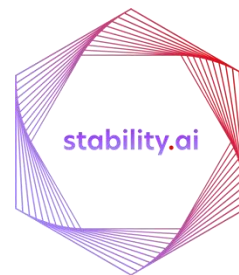
Recent text-to-image models

Let's explore some of the state-of-the-art diffusion models in practice:

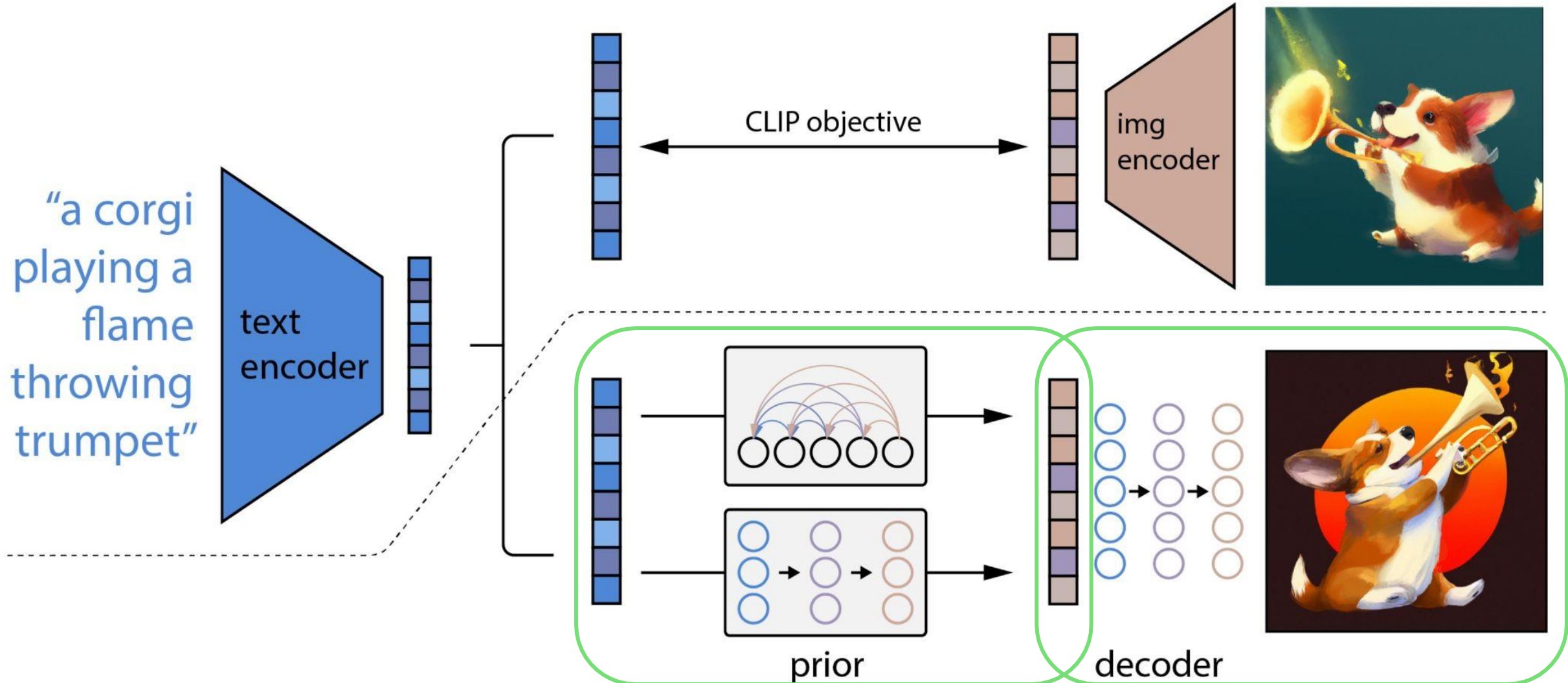
- DALL-E 2  **OpenAI**

- ImageN  **Google Brain**

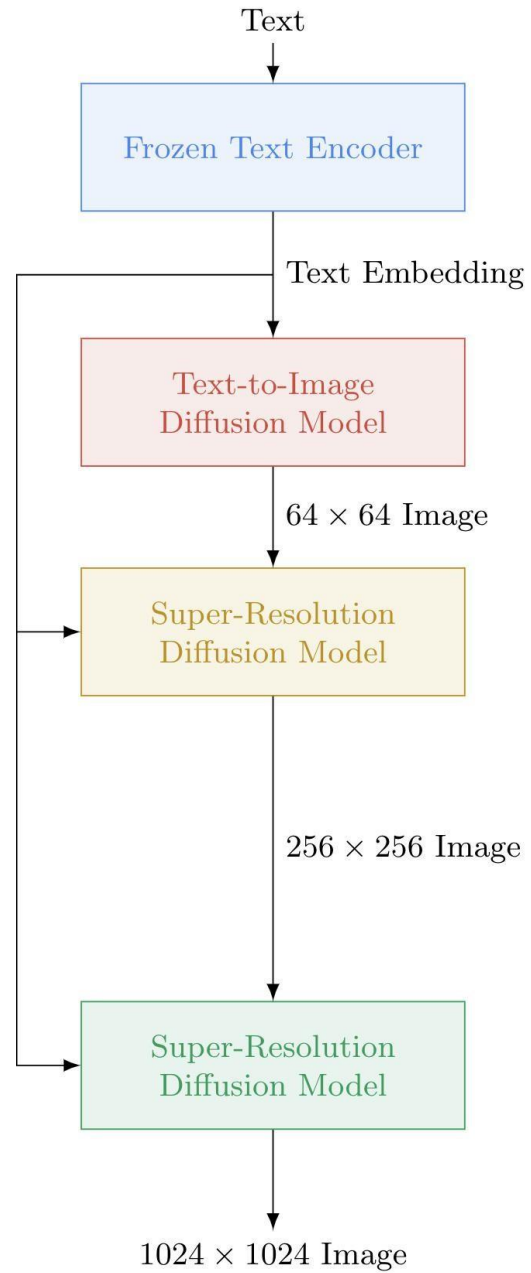
- StableDiffusion  **NVIDIA**



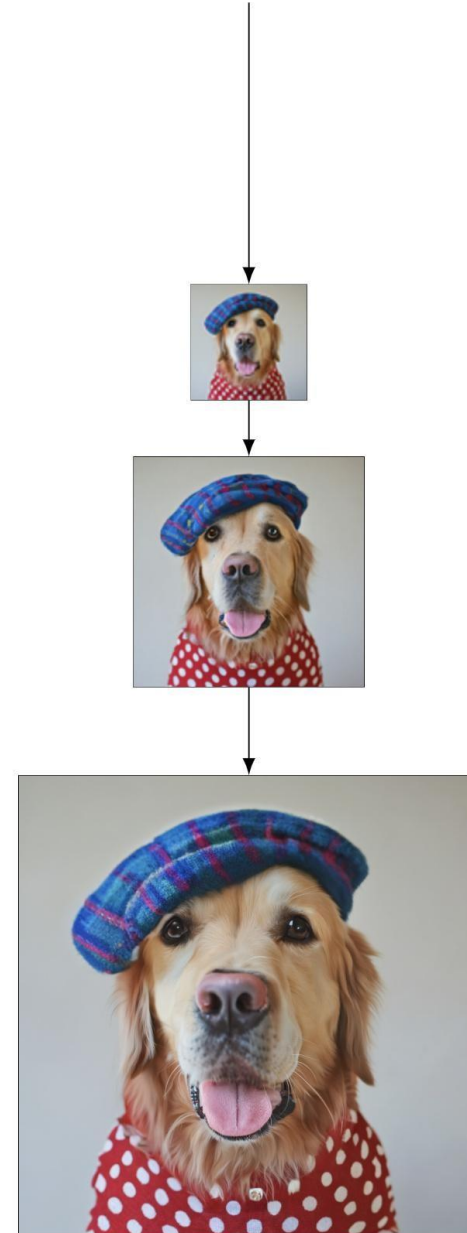
DALL-E 2



ImageN



“A Golden Retriever dog wearing a blue checkered beret and red dotted turtleneck.”



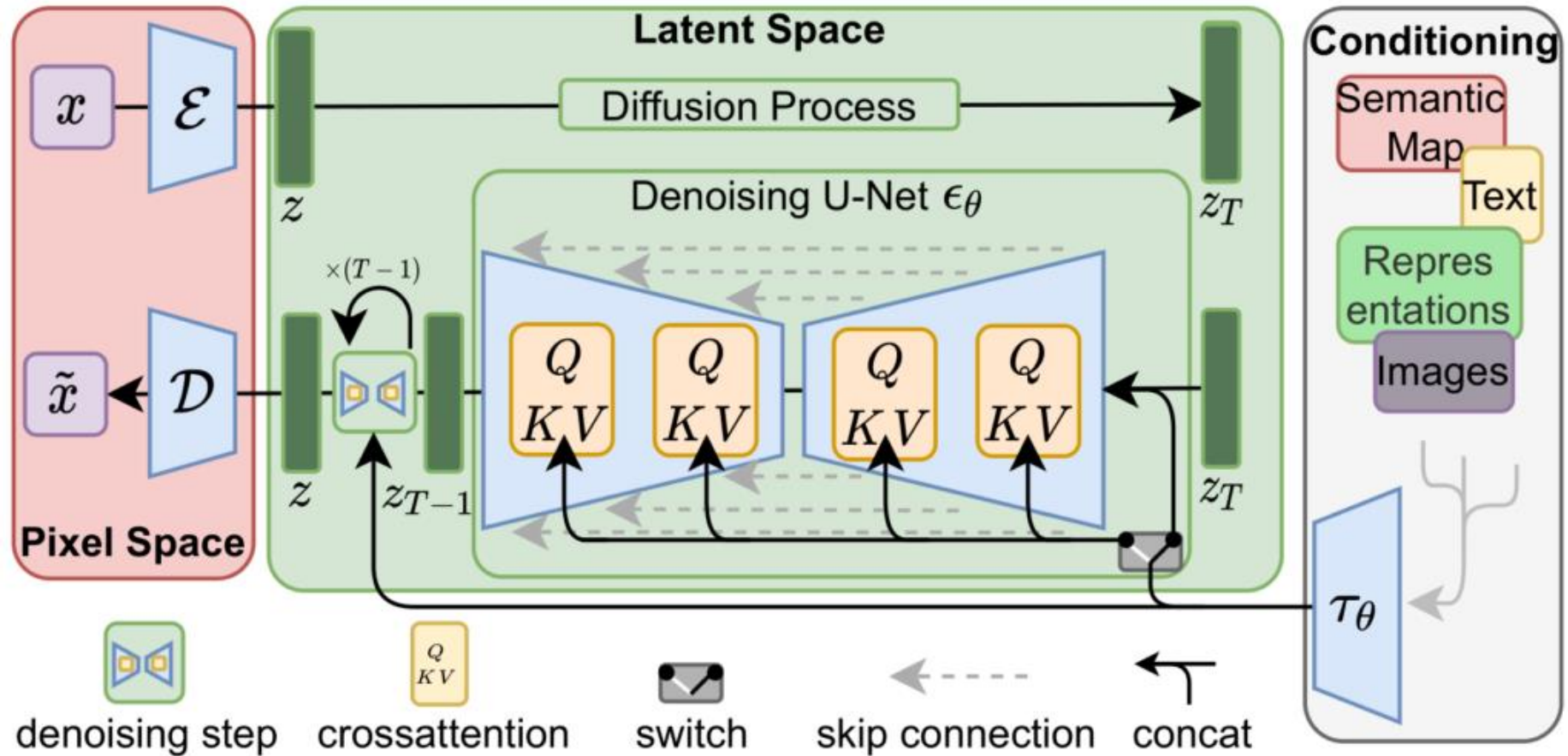
source: [Photorealistic Text-to-Image Diffusion Models with Deep Language Understanding](#)

Latent Diffusion Models

Instead of working in the pixel-level:

1. Encode with a compression model (VAE)
2. Perform diffusion process in the latent space
3. Reconstruct the image from the latent

Stable diffusion

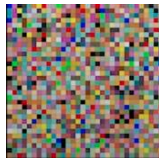


More recent compression method

Stable Diffusion

Compression: 8x

512x512 $\rightarrow$ 64x64



Stable Cascade (Würstchen)

Compression: 12x12

512x512 $\rightarrow$ 12x12



Summary up until now

- **Class-conditioned:** Generates an image with class conditioning. This requires a classifier that has been trained on noisy images.
- **Classifier-free :** Adds conditioning; requires labels.
- **Latent Diffusion Model (Stable Diffusion):** Applies the diffusion process in the latent space

More next lecture